WPI

CS 534
Artificial Intelligence

Week 7: Deep Learning I

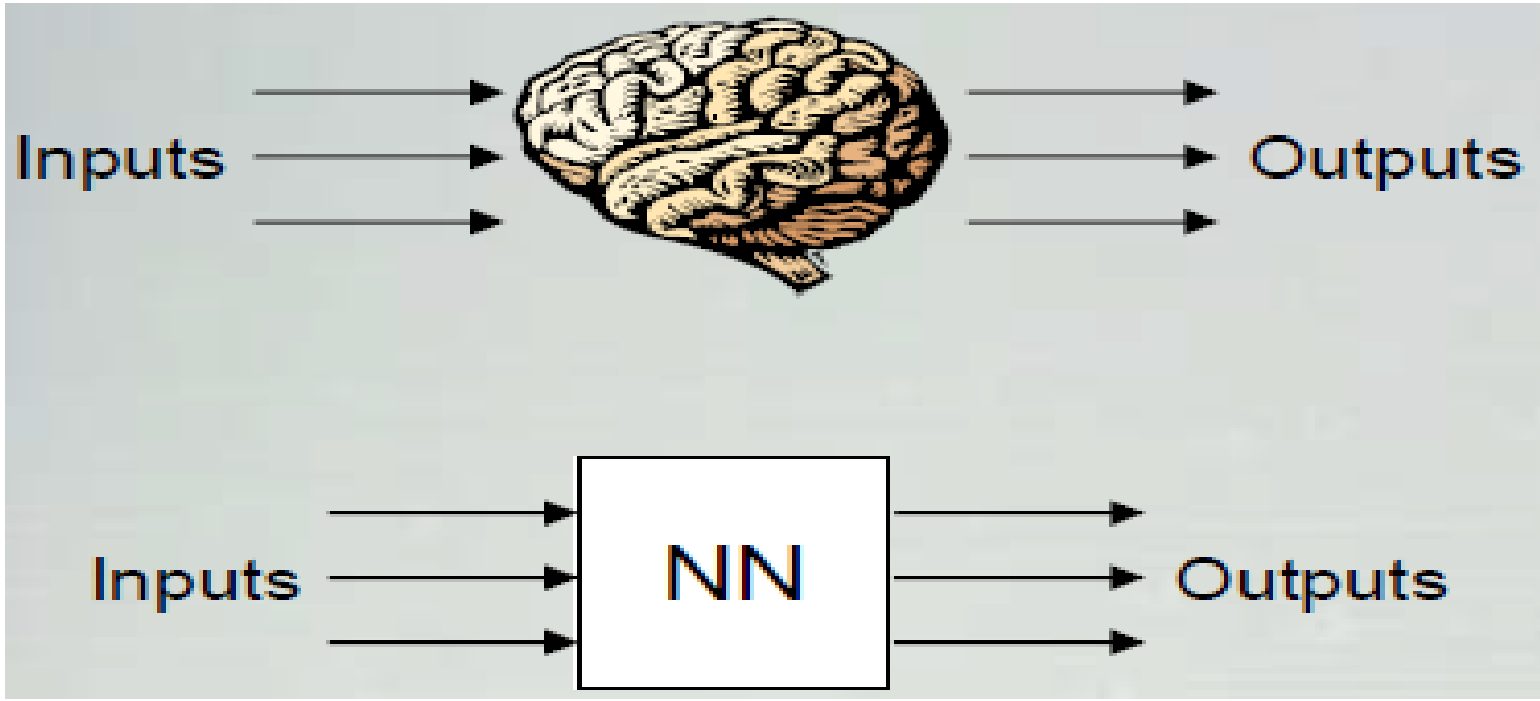
By

Ben C.K. Ngan

Worcester Polytechnic Institute

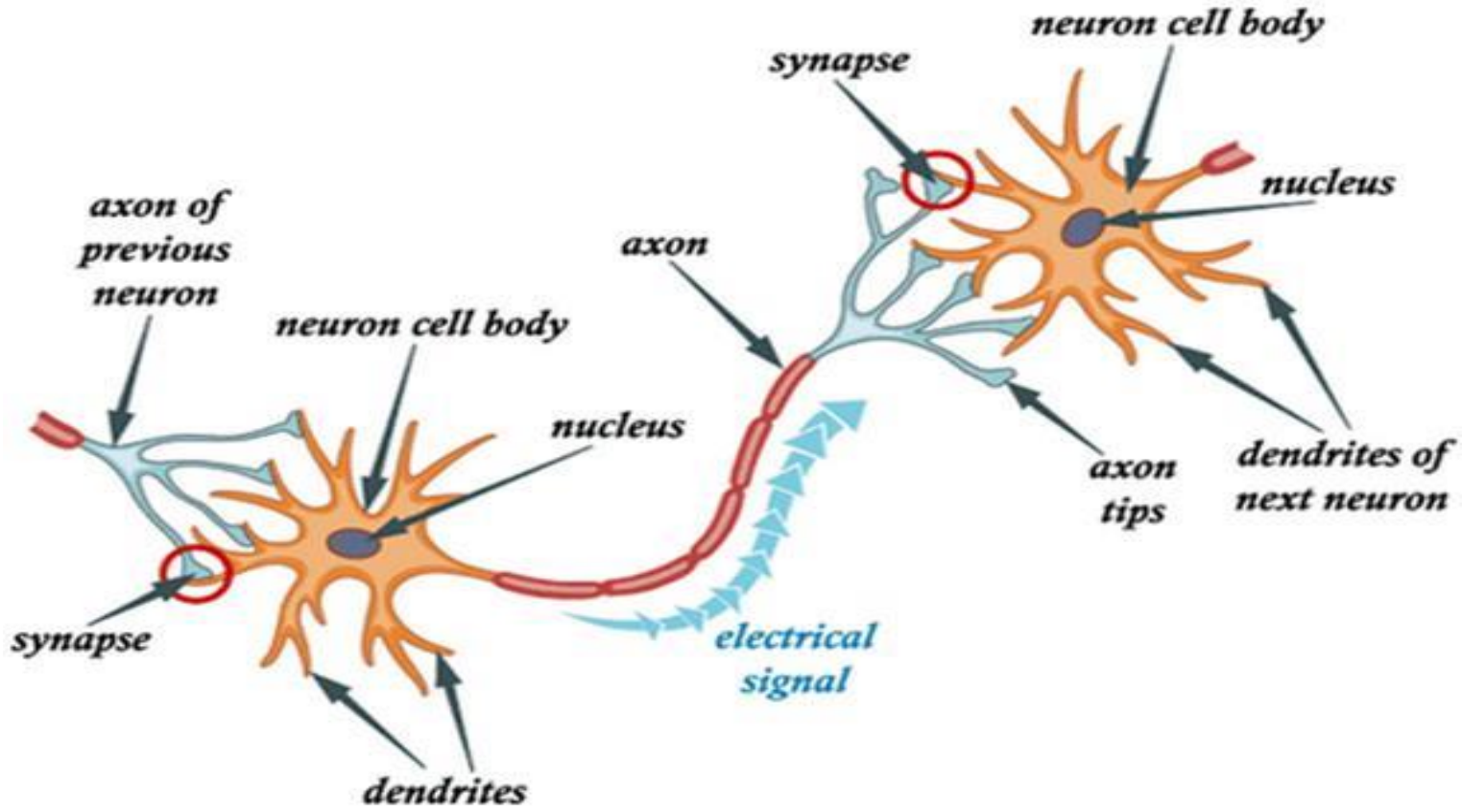
Artificial Neural Network (ANN) Model

- **What is an Artificial Neural Network Model?**
  - An information processing paradigm is inspired by the biological nervous systems to process information.
  - A large number of highly interconnected processing elements, i.e., human neurons, working in parallel to solve a specific problem.



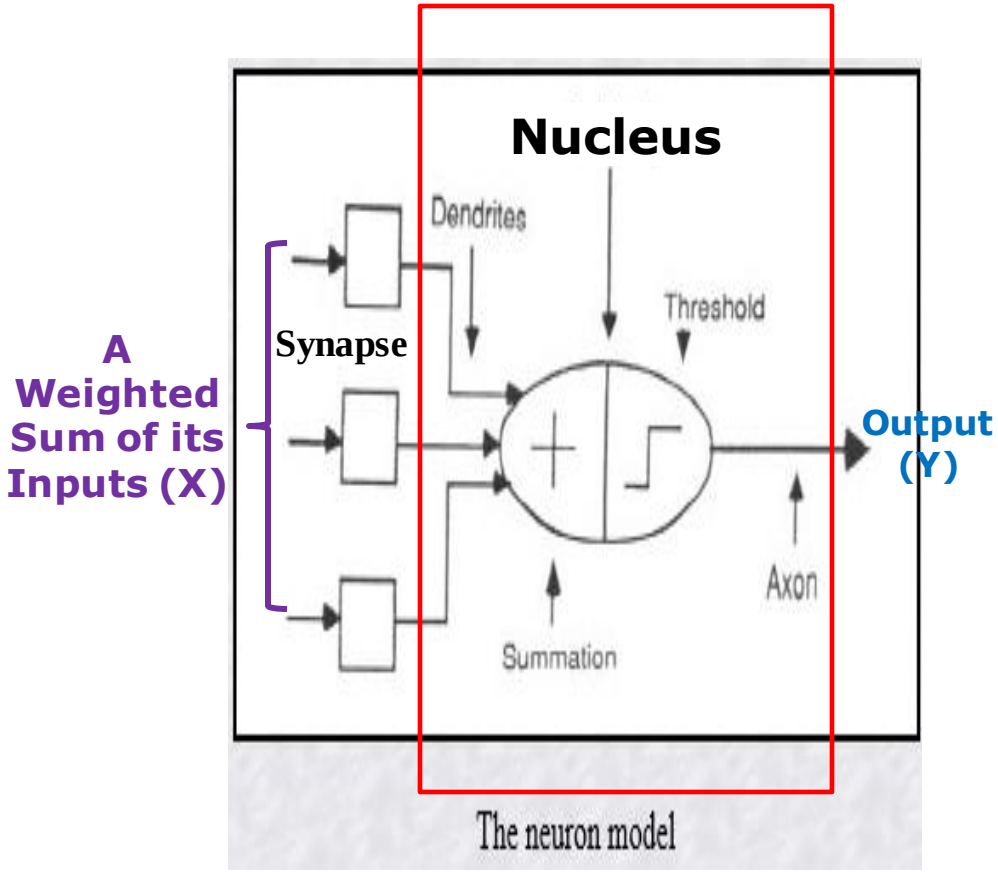
Artificial Neural Network (ANN) Model

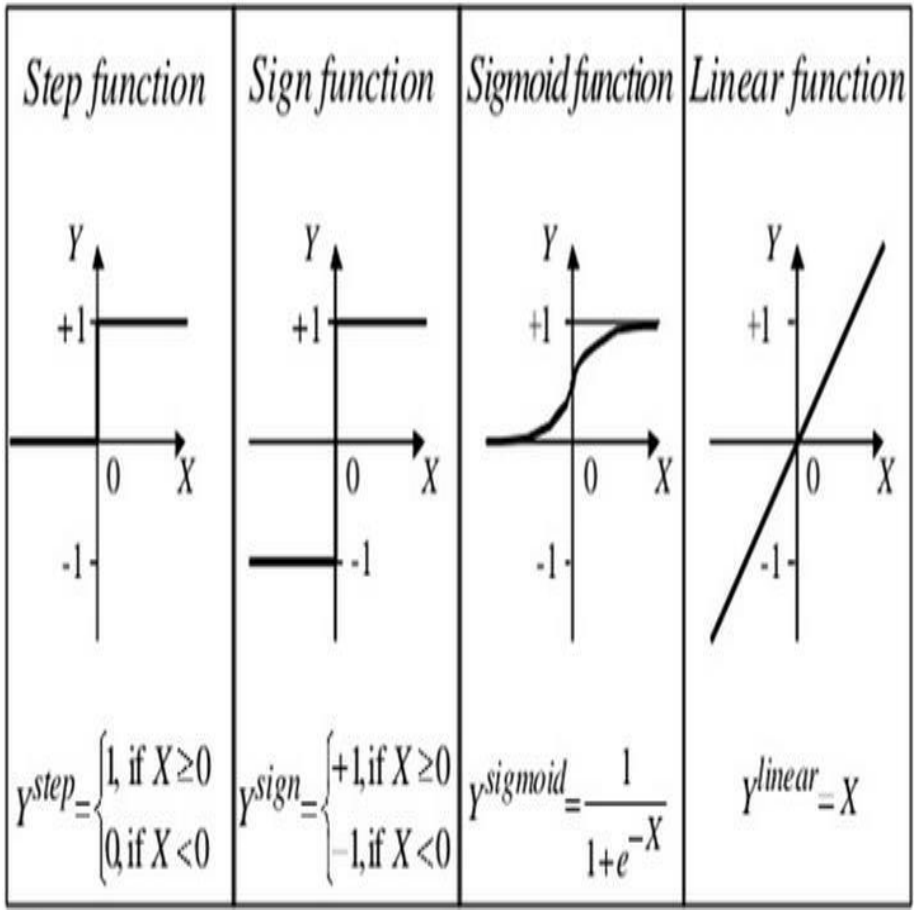
- **What is a human neuron?**
  - A human neuron is composed of dendrites, a nucleus, an axon, and synapses.



Artificial Neural Network (ANN) Model

- **What is an artificial neuron?**
  - An artificial neuron also contains a nucleus, dendrites, one axon, and synapses

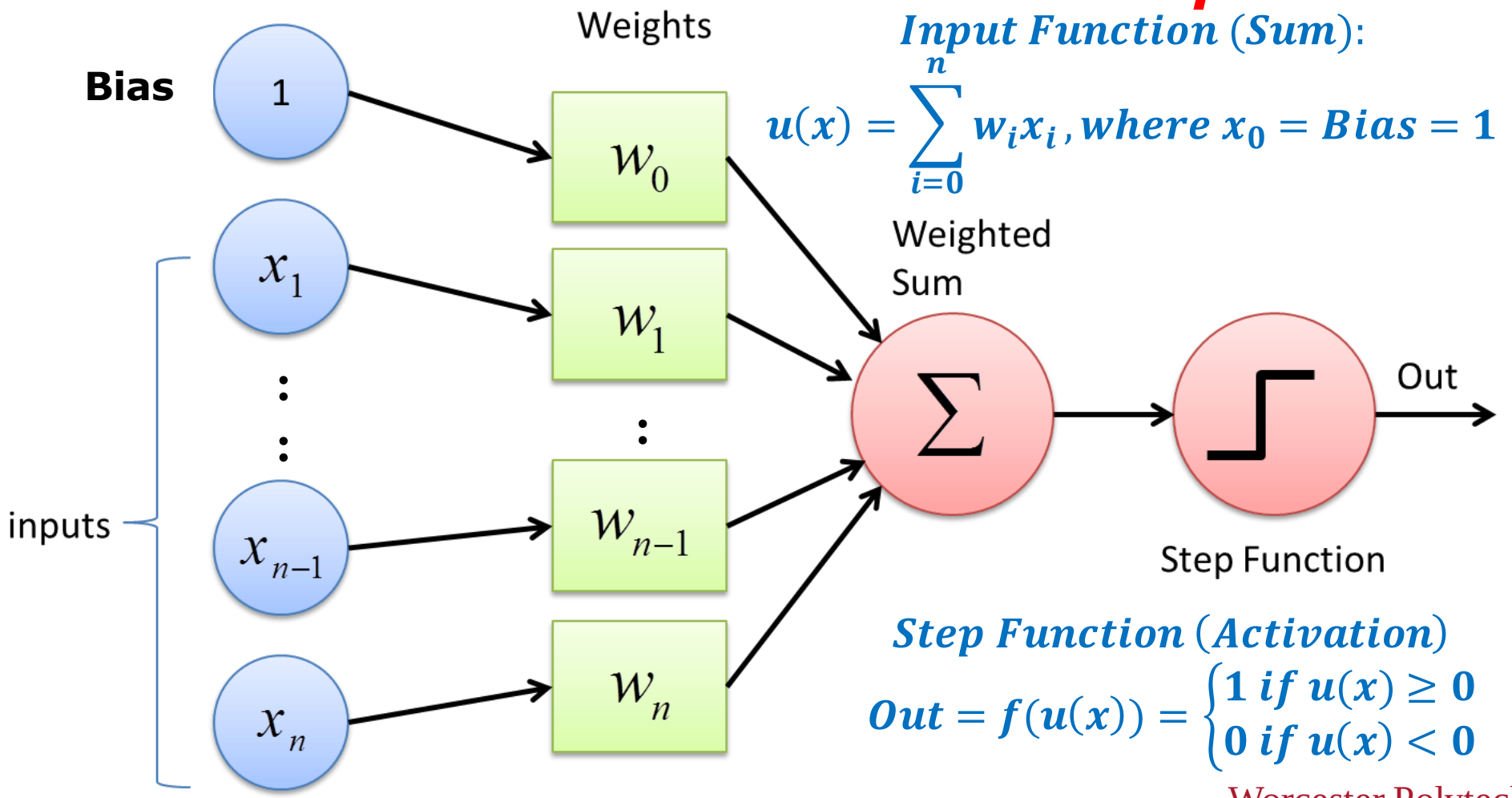




Artificial Neural Network (ANN) Model

- **What is an artificial neuron?**

Perceptron



Input Function (Sum):

$$u(x) = \sum_{i=0}^{n} w_i x_i \, , where \; x_0 = Bias = 1$$

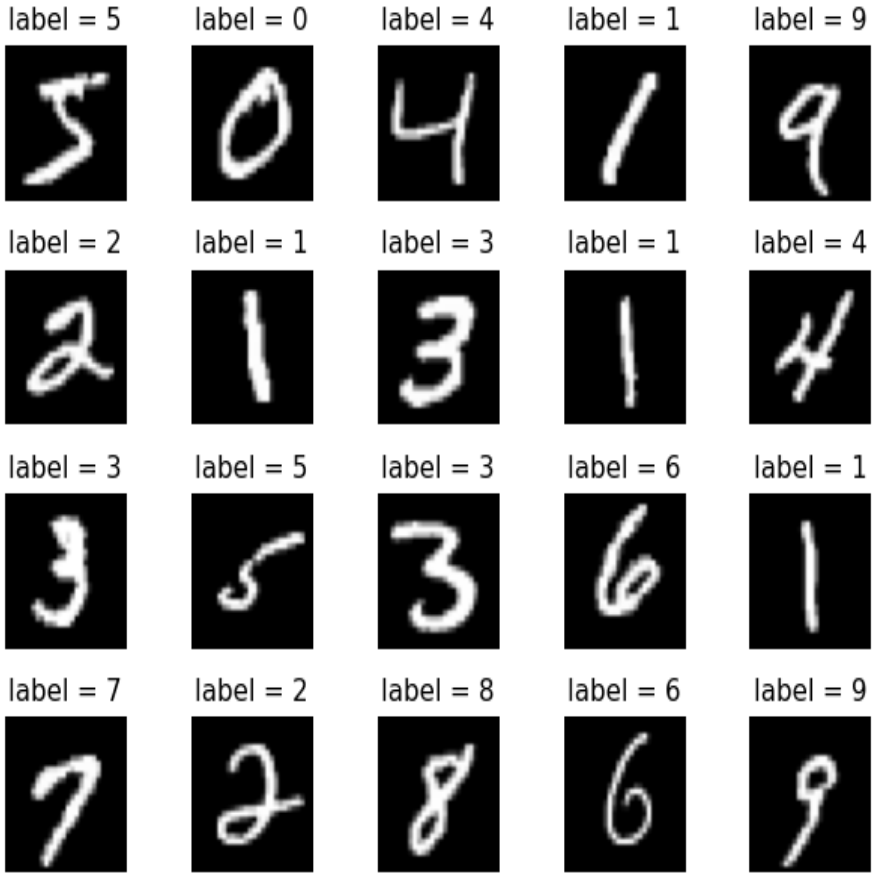
Step Function (Activation)

$$Out = f(u(x)) = \begin{cases} 1 \; if \; u(x) \geq 0 \\ 0 \; if \; u(x) < 0 \end{cases}$$

Hands-on Example: Perceptron

```python
from sklearn.datasets import fetch_openml
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import chainer

def main():
    # Load the MNIST dataset, same as mnist_784, from pre-inn chainer method to plot how images look like.
    train, test = chainer.datasets.get_mnist(ndim=1)
    ROW = 4
    COLUMN = 5
    for i in range(ROW * COLUMN):
        # train[i][0] is i-th image data with size 28x28
        image = train[i][0].reshape(28, 28)  # not necessary to reshape if ndim is set to 2
        plt.subplot(ROW, COLUMN, i + 1)  # subplot with size (width 3, height 5)
        plt.imshow(image, cmap='gray')  # cmap='gray' is for black and white picture.
        # train[i][1] is i-th digit label
        plt.title('label = {}'.format(train[i][1]))
        plt.axis('off')  # do not show axis value
    plt.tight_layout()  # automatic padding between subplots
    plt.savefig('mnist_plot.png')
    plt.show()

    # load and partition MNIST dataset
    X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
    # you can check how if we change random_state (seed for test/train split)
    # the accuracy of our models also change!

    X_train, X_test, y_train, y_test = train_test_split(X/255., y, test_size = 0.20, random_state=1)

    # first let's use a very simple linear perceptron: we set the hyperparams and train
    # a perceptron does not use activation units which means it's a completely linear model
    per = Perceptron(random_state=1, max_iter=50, tol=0.005)
```

Handwritten digit image: This is gray scale image with **size 28 x 28 pixel**.
Label: This is actual digit number this handwritten digit image represents. It is from **0 to 9**.



Hands-on Example: Perceptron

```python
from sklearn.datasets import fetch_openml
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import chainer

def main():
    # Load the MNIST dataset, same as mnist_784, from pre-inn chainer method to plot how images look like.
    train, test = chainer.datasets.get_mnist(ndim=1)
    ROW = 4
    COLUMN = 5
    for i in range(ROW * COLUMN):
        # train[i][0] is i-th image data with size 28x28
        image = train[i][0].reshape(28, 28)  # not necessary to reshape if ndim is set to 2
        plt.subplot(ROW, COLUMN, i + 1)  # subplot with size (width 3, height 5)
        plt.imshow(image, cmap='gray')  # cmap='gray' is for black and white picture.
        # train[i][1] is i-th digit label
        plt.title('label = {}'.format(train[i][1]))
        plt.axis('off')  # do not show axis value
    plt.tight_layout()  # automatic padding between subplots
    plt.savefig('mnist_plot.png')
    plt.show()

    # load and partition MNIST dataset
    X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
    # you can check how if we change random_state (seed for test/train split)
    # the accuracy of our models also change!

    X_train, X_test, y_train, y_test = train_test_split(X/255., y, test_size = 0.20, random_state=1)

    # first let's use a very simple linear perceptron: we set the hyperparams and train
    # a perceptron does not use activation units which means it's a completely linear model
    per = Perceptron(random_state=1, max_iter=50, tol=0.005)
```

version : *int or 'active', default='active'*

Version of the dataset. Can only be provided if also `name` is given. If 'active' the oldest version that's still active is used. Since there may be more than one active version of a dataset, and those versions may fundamentally be different from one another, setting an exact version is highly recommended.

return_X_y : *bool, default=False*

If True, returns `(data, target)` instead of a Bunch object. See below for more information about the `data` and `target` objects.

0 to 255

For a grayscale or b&w image, we have pixel values ranging from 0 to 255. Mar 16, 2021

random_state : *int, RandomState instance or None, default=None*

Controls the shuffling applied to the data before applying the split. Pass an int for reproducible output across multiple function calls. See Glossary.

max_iter : *int, default=1000*

The maximum number of passes over the training data (aka epochs). It only impacts the behavior in the `fit` method, and not the `partial_fit` method.

tol : *float or None, default=1e-3*

The stopping criterion. If it is not None, the iterations will stop when (loss > previous_loss - tol).

n_iter_no_change : *int, default=5*

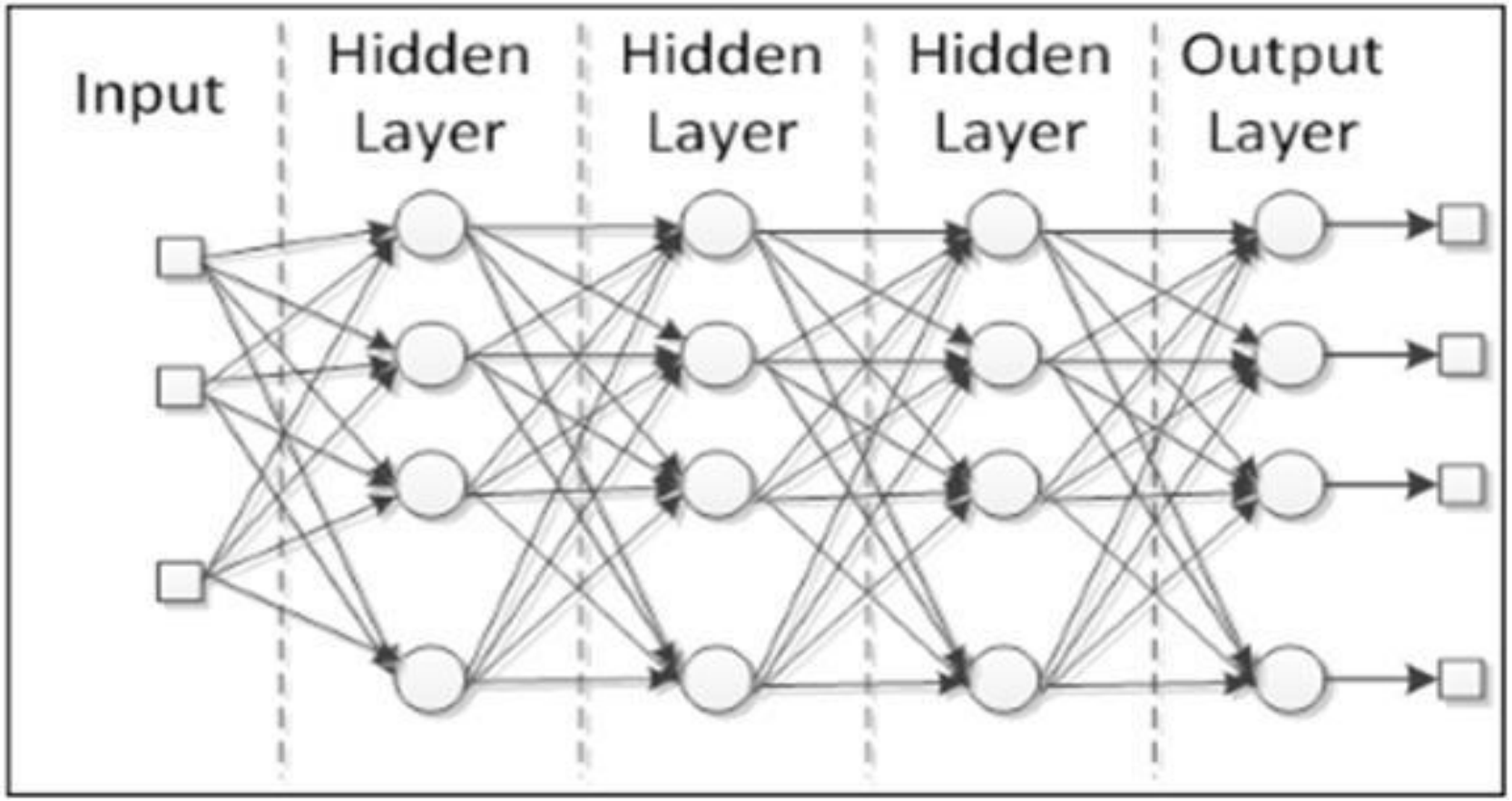
Number of iterations with no improvement to wait before early stopping.

Hands-on Example: Perceptron

```python
from sklearn.datasets import fetch_openml
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import chainer

def main():
    # Load the MNIST dataset, same as mnist_784, from pre-inn chainer method to plot how images look like.
    train, test = chainer.datasets.get_mnist(ndim=1)
    ROW = 4
    COLUMN = 5
    for i in range(ROW * COLUMN):
        # train[i][0] is i-th image data with size 28x28
        image = train[i][0].reshape(28, 28)  # not necessary to reshape if ndim is set to 2
        plt.subplot(ROW, COLUMN, i + 1)  # subplot with size (width 3, height 5)
        plt.imshow(image, cmap='gray')  # cmap='gray' is for black and white picture.
        # train[i][1] is i-th digit label
        plt.title('label = {}'.format(train[i][1]))
        plt.axis('off')  # do not show axis value
    plt.tight_layout()  # automatic padding between subplots
    plt.savefig('mnist_plot.png')
    plt.show()

    # load and partition MNIST dataset
    X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
    # you can check how if we change random_state (seed for test/train split)
    # the accuracy of our models also change!

    X_train, X_test, y_train, y_test = train_test_split(X/255., y, test_size = 0.20, random_state=1)

    # first let's use a very simple linear perceptron: we set the hyperparams and train
    # a perceptron does not use activation units which means it's a completely linear model
    per = Perceptron(random_state=1, max_iter=50, tol=0.005)
    per.fit(X_train, y_train)

    # we predict with our built perceptron
    yhat_train_per = per.predict(X_train)
    yhat_test_per = per.predict(X_test)

    # Training accuracy is usually higher than testing accuracy.
    print(f"Perceptron: Accuracy in train is %.2f" % (accuracy_score(y_train, yhat_train_per)))
    print(f"Perceptron: Accuracy in test is %.2f" % (accuracy_score(y_test, yhat_test_per)))

if __name__ == "__main__":
    main()
```

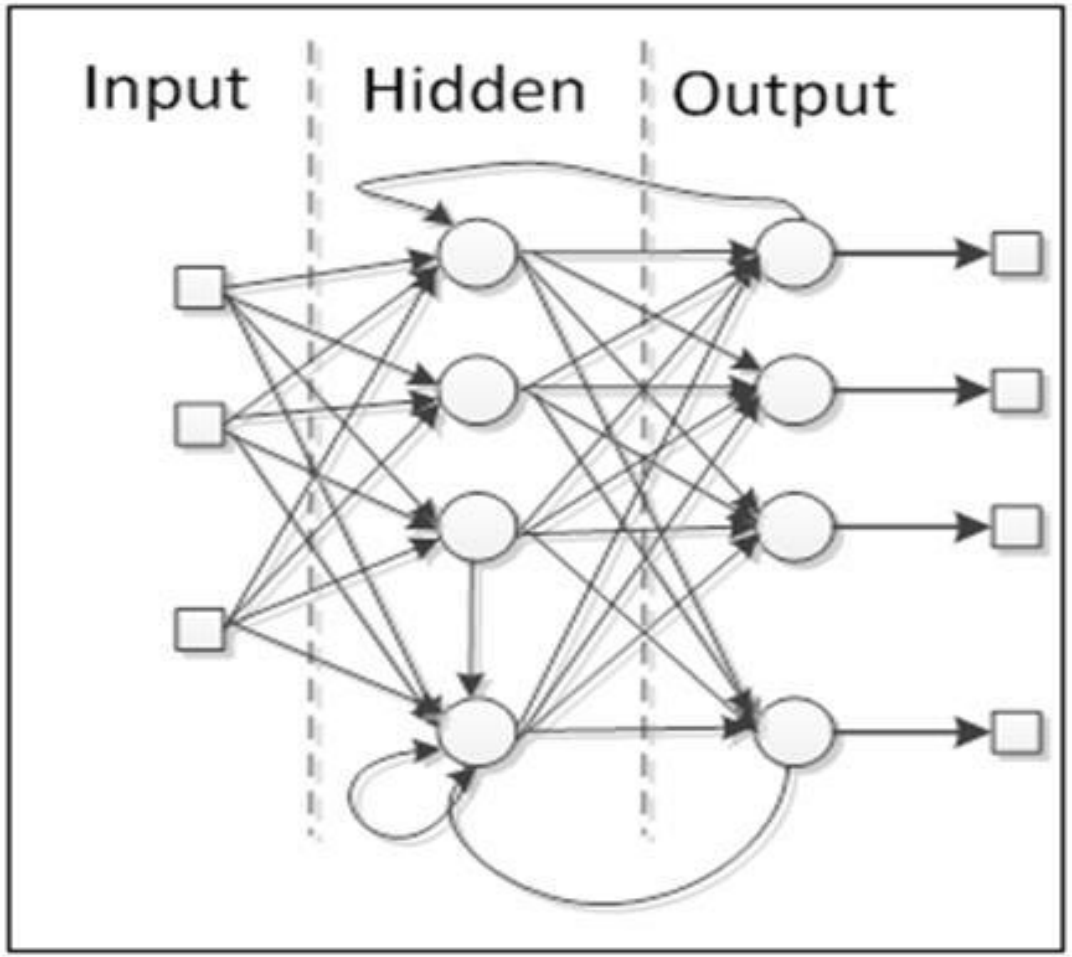
Two Basic Types of ANN Architectures

- **Feedforward** ◯ **A Single Neuron**



Two Basic Types of ANN Architectures

- **Feedback** ◯ **A Single Neuron**



Hands-on Example: Multilayer Perceptron

```python
from sklearn.datasets import fetch_openml
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def main():
    # Load and partition MNIST dataset
    X, y = fetch_openml('mnist_784', version=1, return_X_y=True)

    # Create partitions of training and testing
    X_train, X_test, y_train, y_test = train_test_split(X/255., y, test_size = 0.20, random_state=1)

    # Build an MLP network
    mlp = MLPClassifier(solver="sgd", max_iter=50, verbose=True, random_state=1, learning_rate_init=0.1
                        ,hidden_layer_sizes=(784, 100, 2))
    # Three hidden layers: 784 neurons, 100 neurons, and 2 neurons
    mlp.fit(X_train, y_train)

    # we predict with our built MLP network
    yhat_train_mlp = mlp.predict(X_train)
    yhat_test_mlp = mlp.predict(X_test)

    # Training accuracy is usually higher than testing accuracy.
    print(f"Multilayer Perceptron: Accuracy in train is %.2f" % (accuracy_score(y_train, yhat_train_mlp)))
    print(f"Perceptron: Accuracy in test is %.2f" % (accuracy_score(y_test, yhat_test_mlp)))

if __name__ == "__main__":
    main()
```

hidden_layer_sizes : *array-like of shape(n_layers - 2,), default=(100,)*

The ith element represents the number of neurons in the ith hidden layer.

activation : *{'identity', 'logistic', 'tanh', 'relu'}, default='relu'*

Activation function for the hidden layer.

- 'identity', no-op activation, useful to implement linear bottleneck, returns f(x) = x
- 'logistic', the logistic sigmoid function, returns f(x) = 1 / (1 + exp(-x)).
- 'tanh', the hyperbolic tan function, returns f(x) = tanh(x).
- 'relu', the rectified linear unit function, returns f(x) = max(0, x)

solver : *{'lbfgs', 'sgd', 'adam'}, default='adam'*

The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer proposed by Kingma, Died

random_state : *int, RandomState instance, default=None*

Determines random number generation for weights and bias initialization, train-test split if early stopping is used, and batch sampling when solver='sgd' or 'adam'. Pass an int for reproducible results across multiple function calls. See Glossary.

Hands-on Example: Multilayer Perceptron

```python
from sklearn.datasets import fetch_openml
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def main():
    # Load and partition MNIST dataset
    X, y = fetch_openml('mnist_784', version=1, return_X_y=True)

    # Create partitions of training and testing
    X_train, X_test, y_train, y_test = train_test_split(X/255., y, test_size = 0.20, random_state=1)

    # Build an MLP network
    mlp = MLPClassifier(solver="sgd", max_iter=50, verbose=True, random_state=1, learning_rate_init=0.1
                        ,hidden_layer_sizes=(784, 100, 2))
    # Three hidden layers: 784 neurons, 100 neurons, and 2 neurons
    mlp.fit(X_train, y_train)

    # we predict with our built MLP network
    yhat_train_mlp = mlp.predict(X_train)
    yhat_test_mlp = mlp.predict(X_test)

    # Training accuracy is usually higher than testing accuracy.
    print(f"Multilayer Perceptron: Accuracy in train is %.2f" % (accuracy_score(y_train, yhat_train_mlp)))
    print(f"Perceptron: Accuracy in test is %.2f" % (accuracy_score(y_test, yhat_test_mlp)))

if __name__ == "__main__":
    main()
```

activation : *{'identity', 'logistic', 'tanh', 'relu'}, default='relu'*

Activation function for the hidden layer.

verbose : *bool, default=False*

Whether to print progress messages to stdout.

max_iter : *int, default=200*

Maximum number of iterations. The solver iterates until convergence (determined by 'tol') or this number of iterations. For stochastic solvers ('sgd', 'adam'), note that this determines the number of epochs (how many times each data point will be used), not the number of gradient steps.

learning_rate : *{'constant', 'invscaling', 'adaptive'}, default='constant'*

Learning rate schedule for weight updates.

- 'constant' is a constant learning rate given by 'learning_rate_init'.

learning_rate_init : *float, default=0.001*

The initial learning rate used. It controls the step-size in updating the weights. Only used when solver='sgd' or 'adam'.

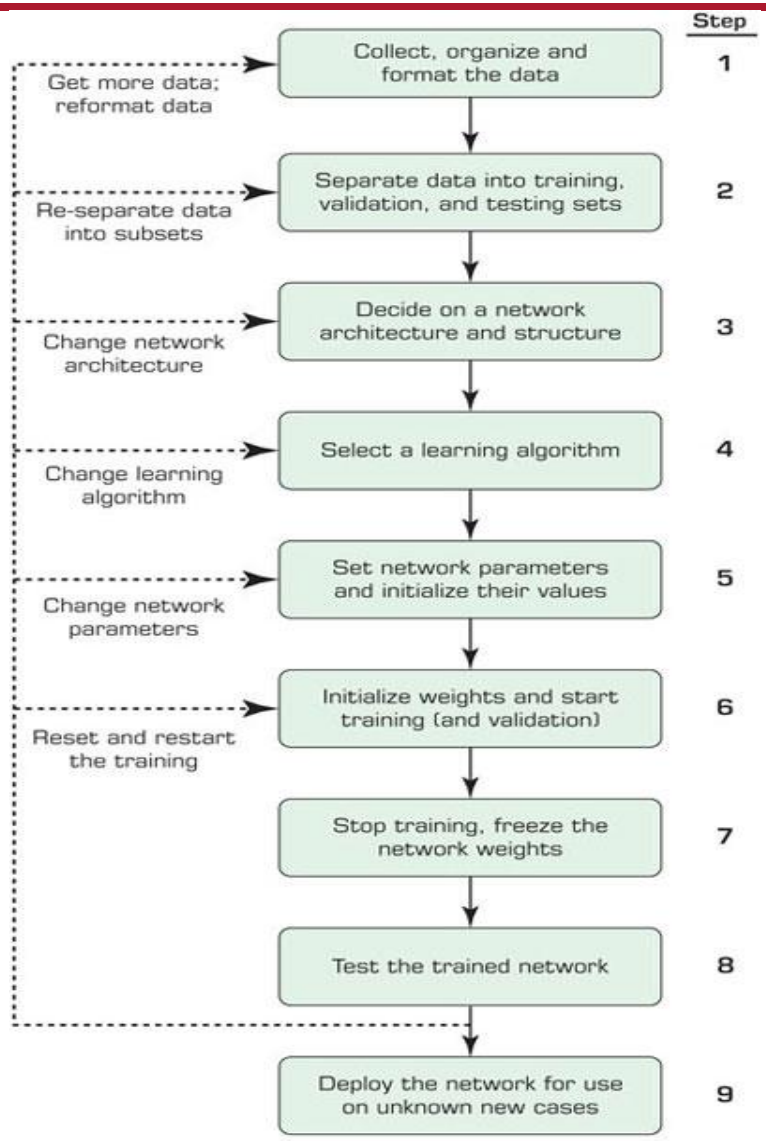
n_iter_no_change : *int, default=10*

Maximum number of epochs to not meet `tol` improvement. Only effective when solver='sgd' or 'adam'.

tol : *float, default=1e-4*

Tolerance for the optimization. When the loss or score is not improving by at least `tol` for `n_iter_no_change` consecutive iterations, unless `learning_rate` is set to 'adaptive', convergence is considered to be reached and training stops.

Development Process of an ANN Model

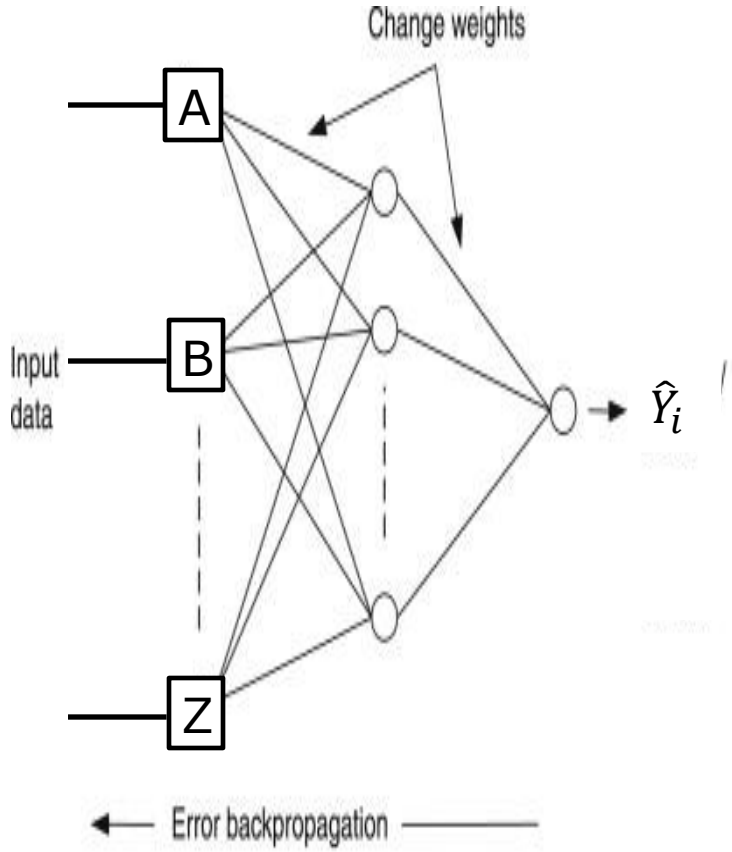


ANN Model Learning Process – Error-correction Principle

- **Supervised Training Process**
  - Define the network architecture (see the example)
  - Initialize the weights (w_0, w_1, …, w_k) on the connections randomly
  - Compute the initial predicted output ($\hat{Y}_i$) on the training datasets
  - Adjust the weights in such a way that the predicted output ($\hat{Y}_i$) of ANN is consistent with actual output (Y_i) of training datasets
  - Objective (Cost) function, e.g., Mean Squared Errors (MSE)

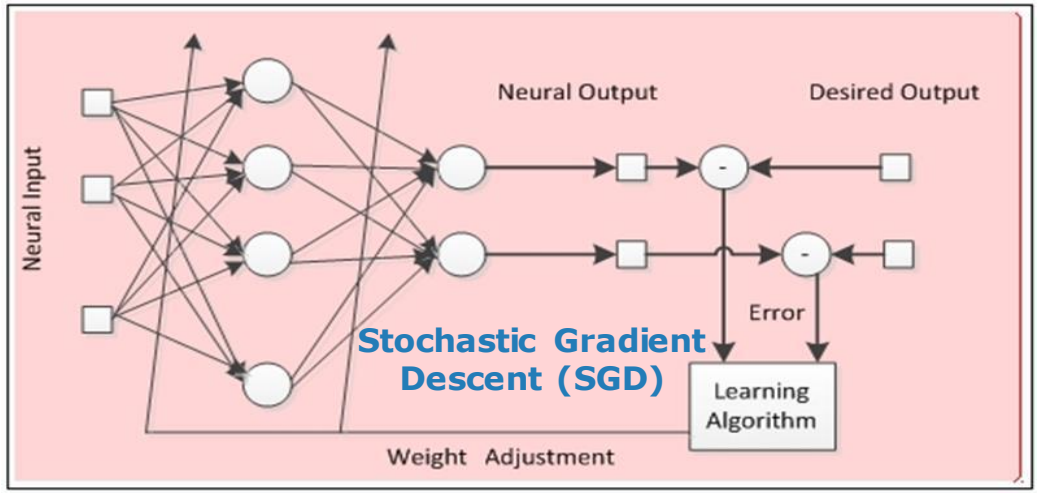
$$\boldsymbol{MSE} = \frac{\mathbf{1}}{\boldsymbol{N}}\sum_{\boldsymbol{i=1}}^{\boldsymbol{N}}(\boldsymbol{Y_i} - \hat{\boldsymbol{Y}}_{\boldsymbol{i}})^{\mathbf{2}}, \textit{where } N \textit{ is the number of training data instances}, Y_i \textit{ is the actual output and } \hat{Y}_i \textit{ is the predicted output}$$

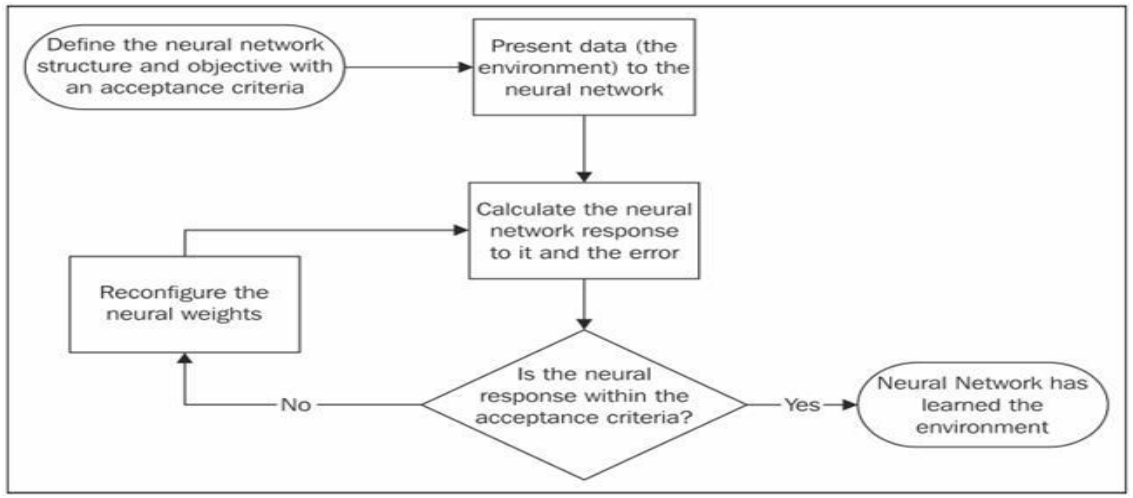
  - Find a set of optimal weights w_i's that minimizes the above objective function, MSE.
  - The back-propagation learning algorithm, i.e., propagate the errors backward, is based on the **error-correction principle**.

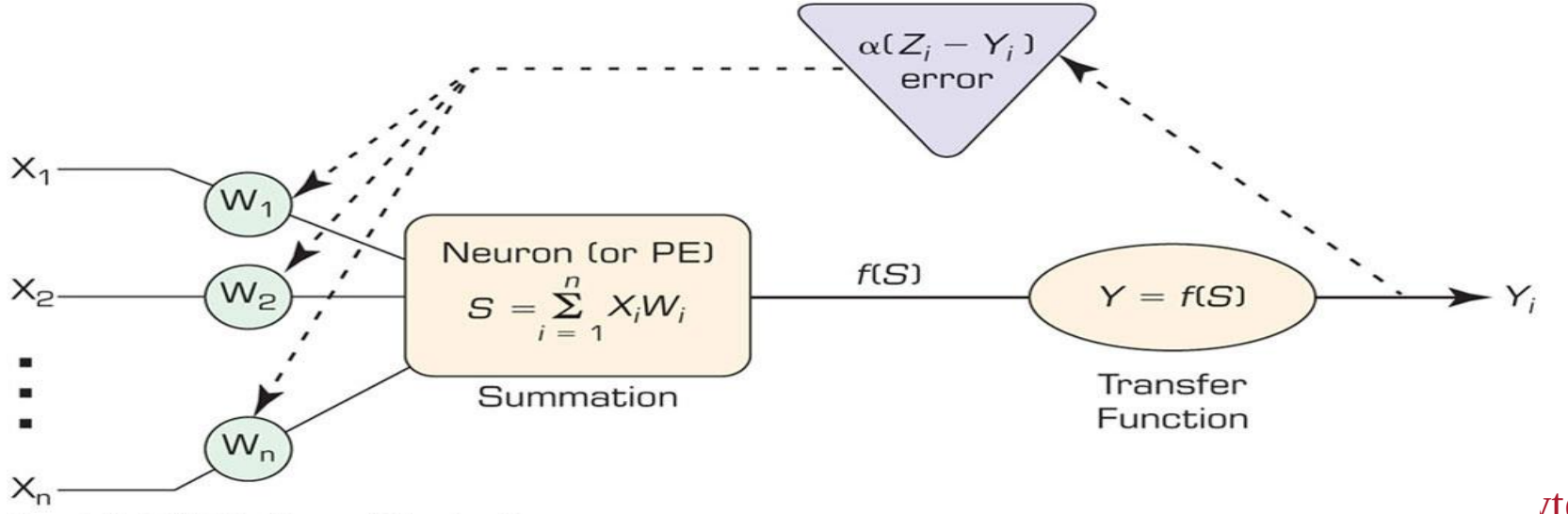


ANN Model Training and Validation Process

- Back-error Propagation for Supervised Training Process

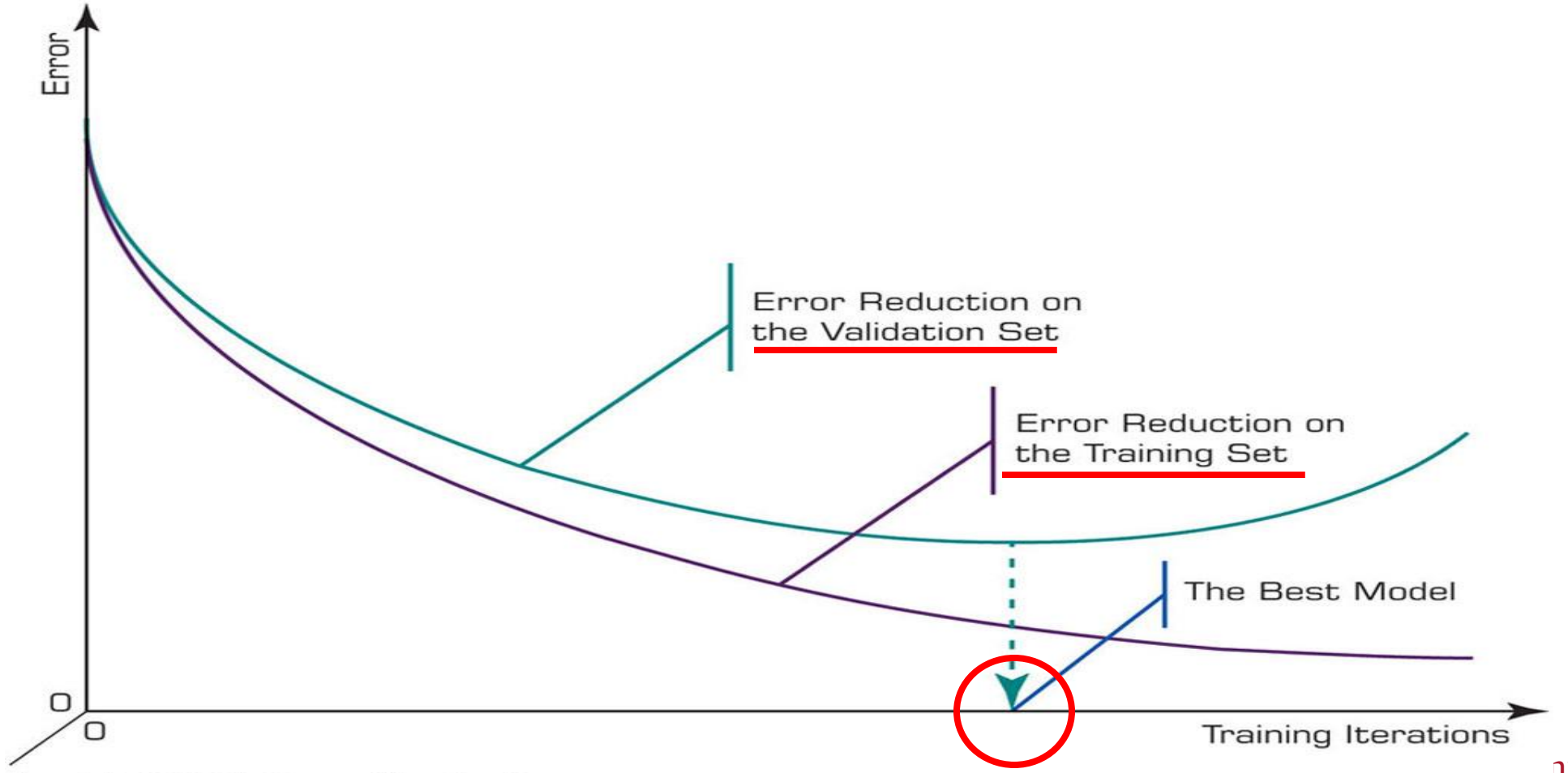






Overfitting in ANN Model

- Whenever the performance stopped improving using the cross-validation, the training process should be stopped.



Overfitting in ANN Model

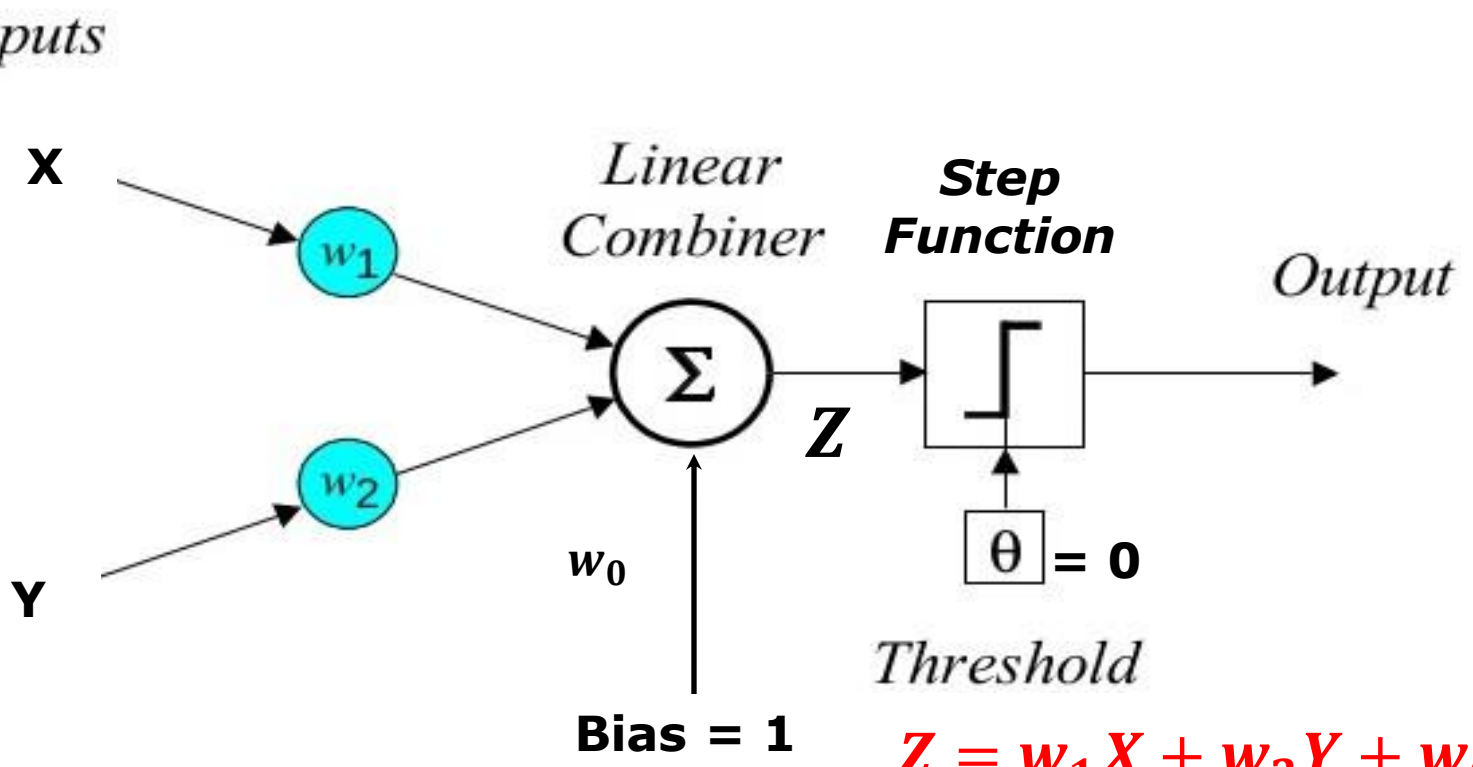
- **Overfitting**: Choose the proper number of nodes in each **hidden layer**. The **more nodes** that are used, the **greater the flexibility** of the neural network, and the greater the likelihood of **overfitting**.

- Some **rule-of-thumb methods** suggested for determining the number of neurons in each hidden layer.
  - The number of neurons in each layer are about **2/3** of the size of the input layer.
  - The number of neurons in each layer should be **twice or less than twice** of the number of neurons in input layer.
  - The number of neurons in each layer is the **average of the input layer size and the output layer size**.

- **Problem**: The above methods are considered to be **suggestions only** because not only the input layer and the output layer decides the size of the hidden layer neurons but also the activation function applied on the neurons, the neural network architecture, the training samples, etc.

Perceptron Learning Algorithm – Logical OR Operator

- A perceptron can be expressed as a linear combination of input values and a bias.

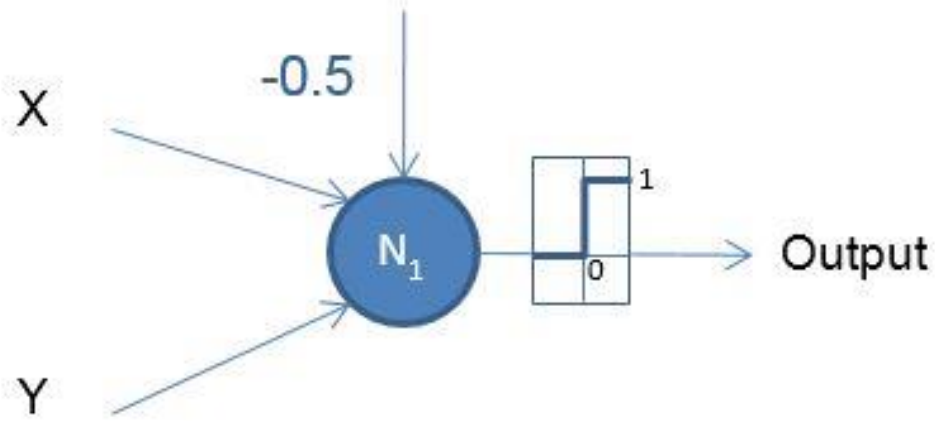


$$Z = w_1 X + w_2 Y + w_0 \cdot Bias$$

$$Output = \begin{cases} 1 \; if \; Z \geq 0 \\ 0 \; if Z < 0 \end{cases}$$

Perceptron Learning Algorithm – Logical OR Operator

- $w_1 = 1$, $w_2 = 1$, and $w_0 = -0.5$

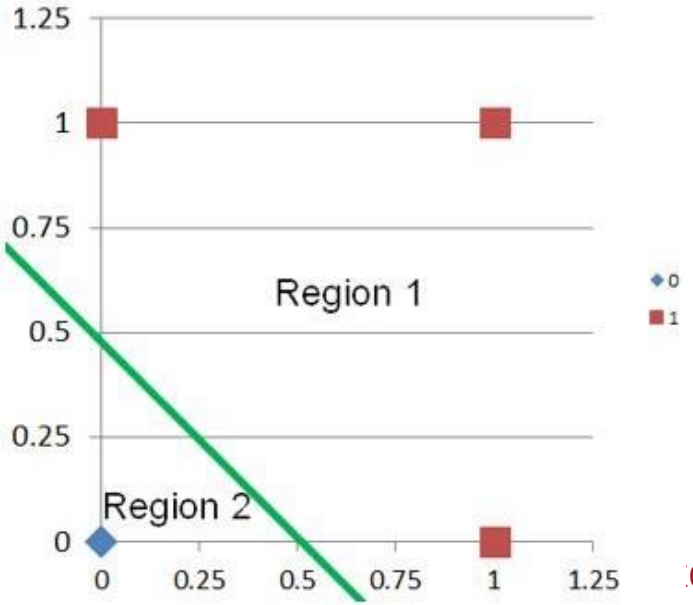


X	Y	Output
0	0	0
0	1	1
1	0	1
1	1	1

$$Z = w_1X + w_2Y + w_0$$

$$Z = X + Y - 0.5$$

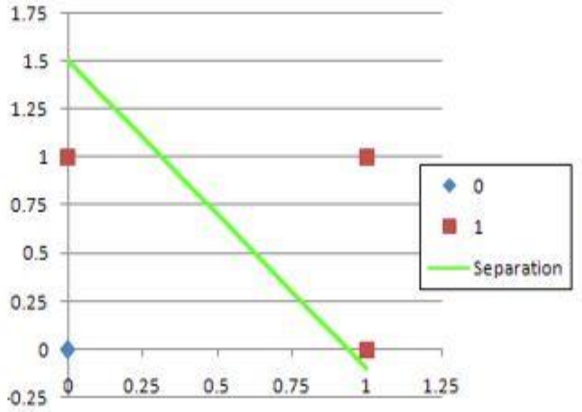
$$Output = \begin{cases} 1 \; if \; Z \geq 0 \\ 0 \; if \; Z < 0 \end{cases}$$

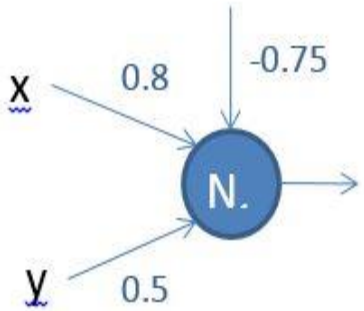


Perceptron Learning Algorithm – Logical OR Operator

- Suppose

$$Z = 0.8X + 0.5Y - 0.75$$





X	Y	Output
0	0	0
0	1	1
1	0	1
1	1	1

1st Epoch:

STEP 1: (0,0)

$Z = 0.8 * 0 + 0.5 * 0 - 0.75 < 0$

Predicted Output = 0 CORRECT!

STEP 2: (0,1)*

$Z = 0.8 * 0 + 0.5 * 1 - 0.75 < 0$

Predicted Output = 0 WRONG!

STEP 3: (1,0)

$Z = 0.8 * 1 + 0.6 * 0 - 0.65 \geq 0$

Predicted Output = 1 CORRECT!

STEP 4: (1,1)

$Z = 0.8 * 1 + 0.6 * 1 - 0.65 \geq 0$

Predicted Output = 1 CORRECT!

Perceptron Learning Algorithm – Logical OR Operator

STEP 2: (0,1)*

$Z = 0.8 * 0 + 0.5 * 1 - 0.75 < 0$

Output = 0 WRONG!

Error
= Output$_{actual}$ – Output$_{predicted}$
= 1 – 0
= 1

$\boldsymbol{\Delta w_i = \alpha * error * i, where\ \alpha\ is\ the\ learning\ rate = 0.1, and\ i = X, Y, and\ Bias}$

X	Y	Output
0	0	0
0	**1**	1
1	0	1
1	1	1

$$w_{x_{new}} = w_{x_{old}} + \Delta w_x = 0.8 + 0.1 * 1 * 0 = 0.8$$
$$w_{y_{new}} = w_{y_{old}} + \Delta w_y = 0.5 + 0.1 * 1 * 1 = 0.6$$
$$w_{Bias_{new}} = w_{Bias_{old}} + \Delta w_{Bias} = -0.75 + 0.1 * 1 * 1 = -0.65$$

$$\boldsymbol{Z = 0.8X + 0.6Y - 0.65}$$

Perceptron Learning Algorithm – Logical OR Operator

- Now $Z = 0.8X + 0.6Y - 0.65$

X	Y	Output
0	0	0
0	1	1
1	0	1
1	1	1

2nd Epoch:

STEP 5: (0,0)

$Z = 0.8 * 0 + 0.6 * 0 - 0.65 < 0$

Predicted Output = 0 CORRECT!

STEP 6: (0,1)*

$Z = 0.8 * 0 + 0.6 * 1 - 0.65 < 0$

Predicted Output = 0 WRONG!

STEP 7: (1,0)

$Z = 0.8 * 1 + 0.7 * 0 - 0.55 \geq 0$

Predicted Output = 1 CORRECT!

STEP 8: (1,1)

$Z = 0.8 * 1 + 0.7 * 1 - 0.55 \geq 0$

Predicted Output = 1 CORRECT!

Perceptron Learning Algorithm – Logical OR Operator

STEP 6: (0,1)*

$Z = 0.8 * 0 + 0.6 * 1 - 0.65 < 0$

Output = 0 WRONG AGAIN !

X	Y	Output
0	0	0
0	**1**	1
1	0	1
1	1	1

Error
= $Output_{actual}$ – $Output_{predicted}$
= 1 – 0
= 1

$\Delta w_i = \alpha * error * i, where\ \alpha\ is\ the\ learning\ rate = 0.1, and\ i = X, Y, and\ Bias$

$$w_{x_{new}} = w_{x_{old}} + \Delta w_x = 0.8 + 0.1 * 1 * 0 = 0.8$$
$$w_{y_{new}} = w_{y_{old}} + \Delta w_y = 0.6 + 0.1 * 1 * 1 = 0.7$$
$$w_{Bias_{new}} = w_{Bias_{old}} + \Delta w_{Bias} = -0.65 + 0.1 * 1 * 1 = -0.55$$

$$Z = 0.8X + 0.7Y - 0.55$$

Perceptron Learning Algorithm – Logical OR Operator

- Now $Z = 0.8X + 0.7Y - 0.55$

X	Y	Output
0	0	0
0	1	1
1	0	1
1	1	1

The algorithm converges (stops) when either:
(1)The training data is linearly separable or
(2)The max epoch has been reached or
(3)The loss has not been improved for several consecutive epochs.

3rd Epoch:

STEP 5: (0,0)

$Z = 0.8 * 0 + 0.7 * 0 - 0.55 < 0$

Predicted Output = 0 CORRECT!

STEP 6: (1,0)

$Z = 0.8 * 1 + 0.7 * 0 - 0.55 \geq 0$

Predicted Output = 1 CORRECT!

STEP 7: (1,1)

$Z = 0.8 * 1 + 0.7 * 1 - 0.55 \geq 0$

Predicted Output = 1 CORRECT!

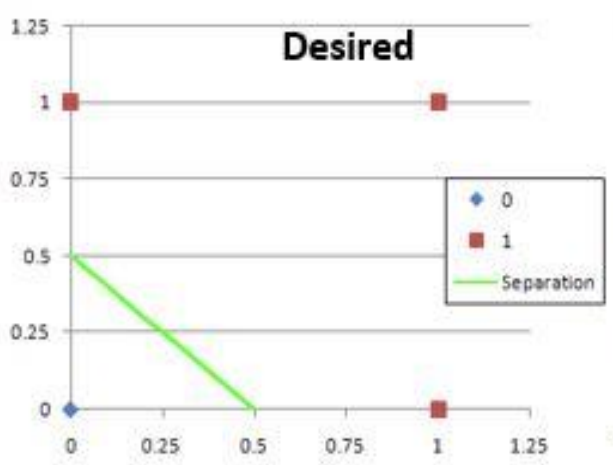
STEP 8: (0,1)

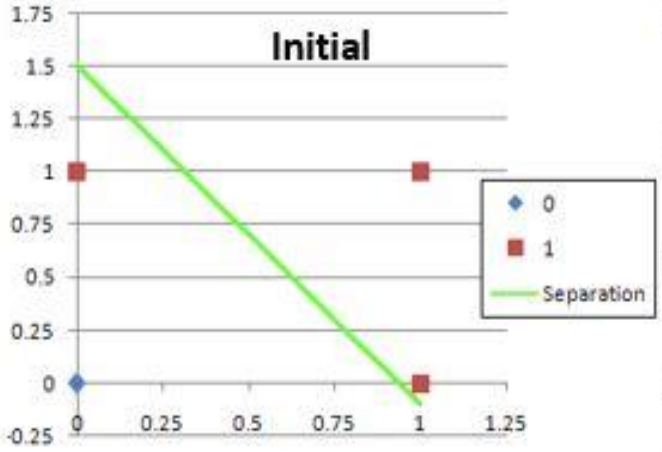
$Z = 0.8 * 0 + 0.7 * 1 - 0.55 \geq 0$

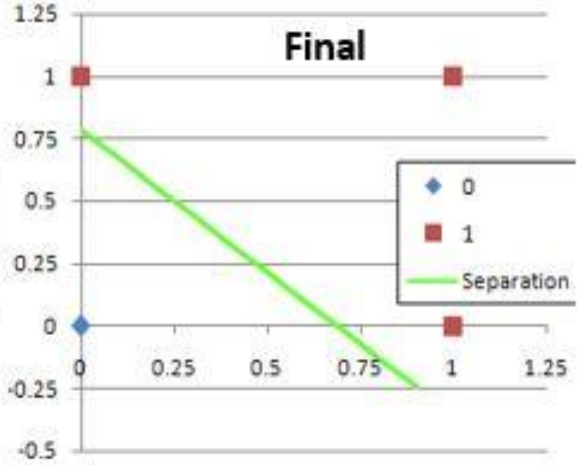
Predicted Output = 1 CORRECT!

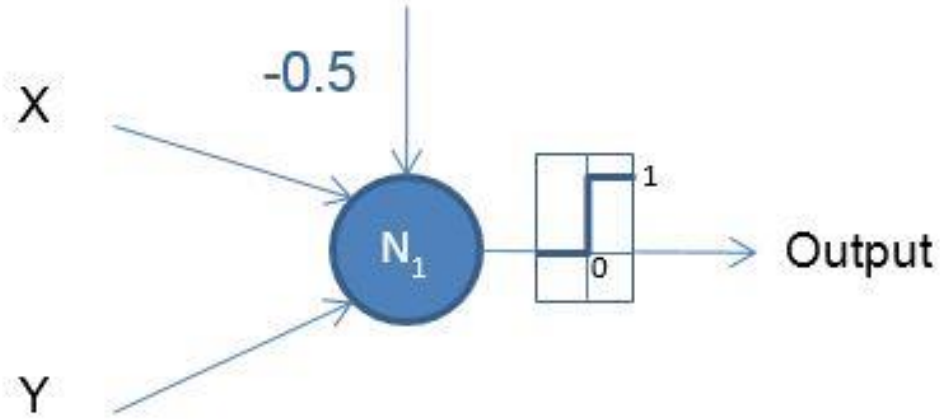
Perceptron Learning Algorithm – Logical OR Operator

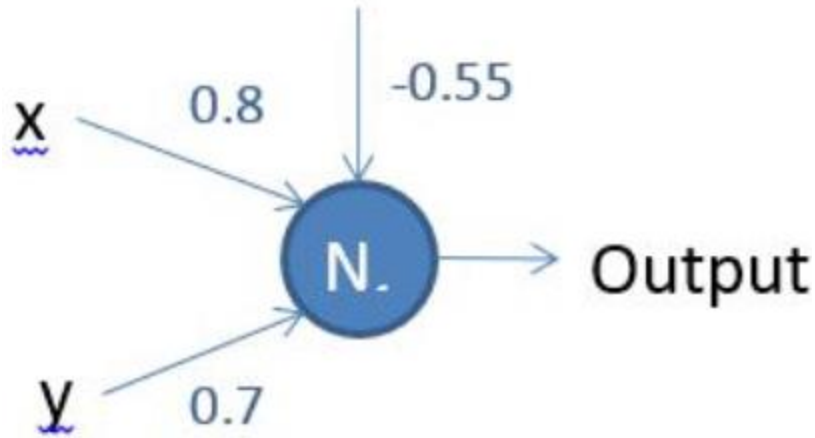
$$Z = 0.8X + 0.7Y - 0.55$$





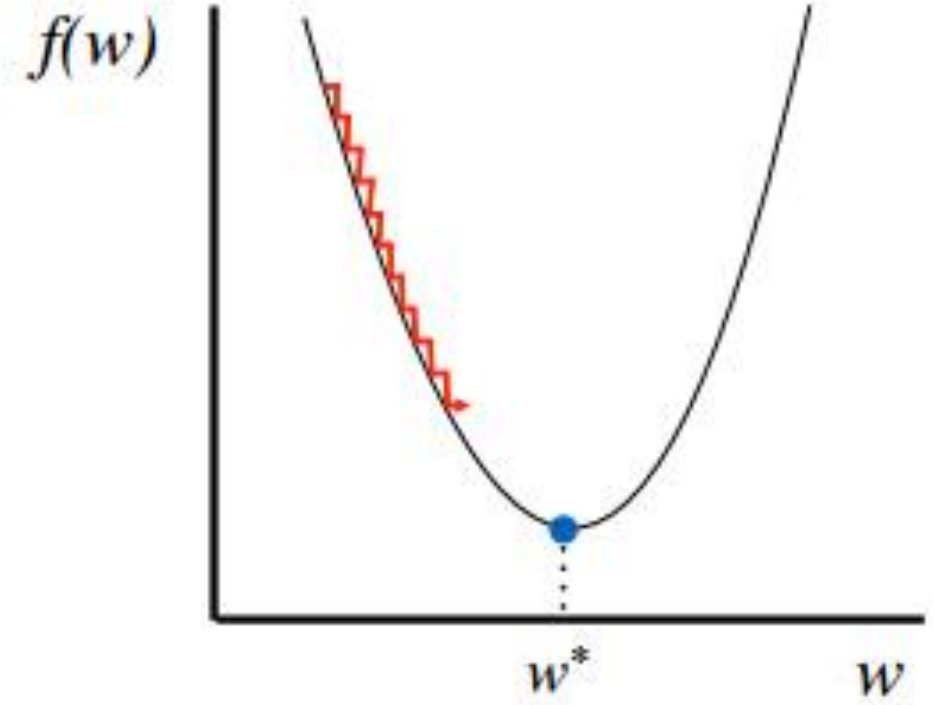




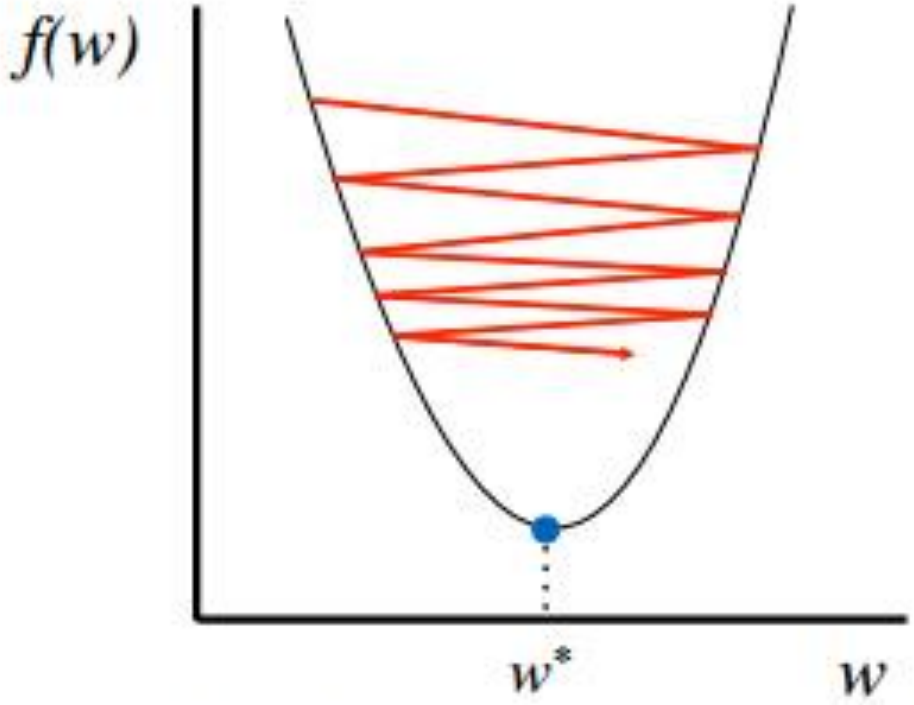


Anil K. Jain, Jianchang Mao, K. M. Mohiuddin. (1996). Artificial Neural Networks: A Tutorial. IEEE Computer.

Learning Rate



Too small: converge very slowly

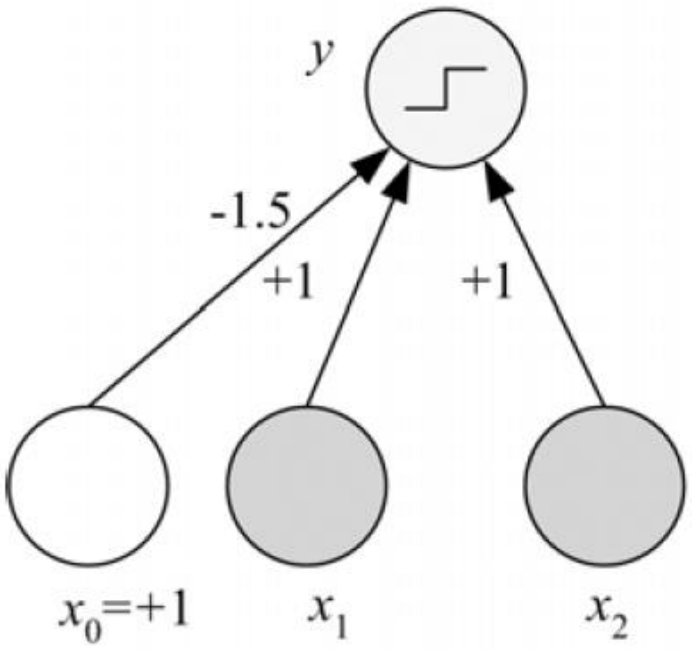


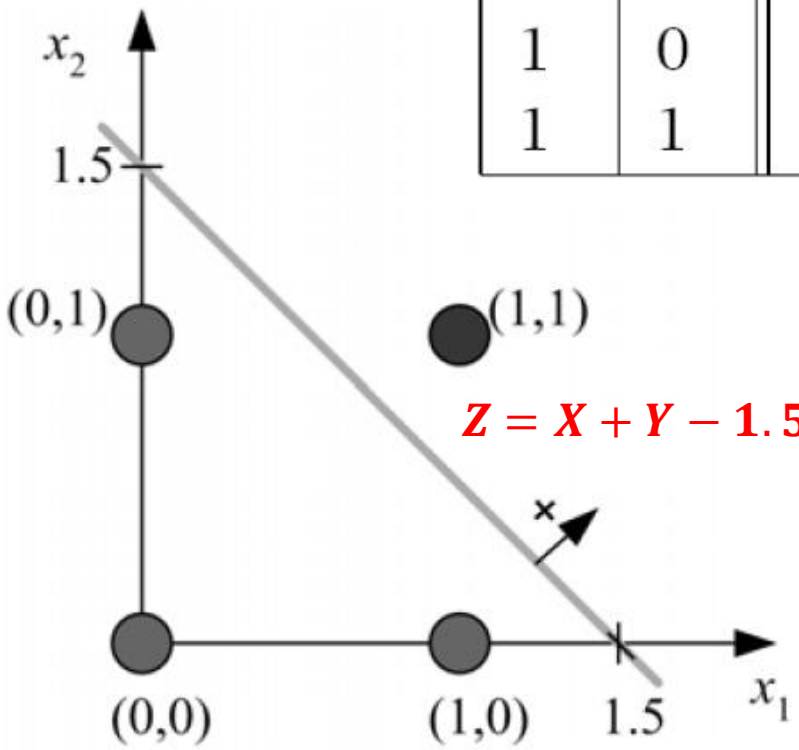
Too big: overshoot and even diverge

Perceptron Learning Algorithm – Logical AND Operator

Learning Boolean AND

x_1	x_2	r
0	0	0
0	1	0
1	0	0
1	1	1





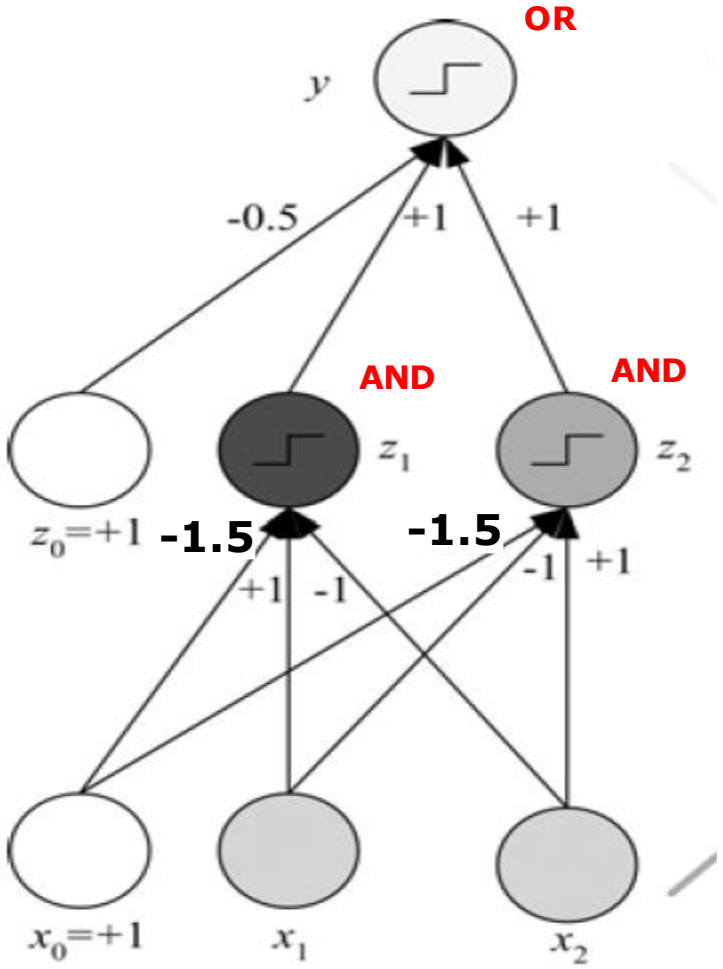
$$Z = X + Y - 1.5$$

Perceptron Learning Algorithm – Logical XOR Operator

XOR

- No v

x_1	x_2	r
0	0	0
0	1	1
1	0	1
1	1	0



z_1 z_2

x_1 XOR x_2 = (x_1 AND ~x_2) OR (~x_1 AND x_2)

Hands-on Example: MLP Classification

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

def main():
    # Load our dataset
    raw_data = "seeds_dataset.txt"
    dataset = np.loadtxt(raw_data)
    print("Dataset:")
    print(dataset)
    print("\nShape of Dataset: ", dataset.shape) # We have 210 data points and the 8 features listed above.

    # Create partitions of training and testing
    # The first seven columns are features and the 8th column is the target
    X_train, X_test, y_train, y_test = train_test_split(dataset[:, 0:7], dataset[:, 7],
                                                        test_size=0.25, random_state=42)

    # 5 neurons in the first hidden layer and 3 in the second one
    clf = MLPClassifier(solver="sgd", alpha=1e-5, hidden_layer_sizes=(5, 3), random_state=1)
    clf.fit(X_train, y_train)

    # Use the predict_proba() function to return the estimated probability of three classes
    prediction_probability = clf.predict_proba(X_test[2:3,:])
    print("\nPrediction Probability: ", prediction_probability)

    # Use the predict() function to predict which class the data points belong to
    prediction_result = clf.predict(X_test[2:3,:])
    print("\nPrediction Result: ", prediction_result) # The prediction result is class 2

    # Use the score() to compute the mean accuracy of the given test data and labels
    acc = clf.score(X_test, y_test)
    print("\nMean Accuracy: ", acc)

if __name__ == "__main__":
    main()
```

7 Input Attributes: area, perimeter, compactness, length of kernel, width of kernel, asymmetry coefficient, length of kernel groove
1 Output Attribute: 3 Classes of Seeds (1, 2, and 3)

alpha : *float, default=0.0001*

Strength of the L2 regularization term. The L2 regularization term is divided by the sample size when added to the loss.

Alpha is a parameter for regularization term, aka penalty term, that combats overfitting by constraining the size of the weights.

Hands-on Example: MLP Regression

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor

def main():
    # Load our dataset
    raw_data = "yacht_hydrodynamics.data"
    dataset = np.loadtxt(raw_data)
    print("Dataset:")
    print(dataset)
    print("\nShape of Dataset: ", dataset.shape) # We have 308 data points and the 7 features listed above.

    # Use the StandardScaler() function to normalize the dataset
    data_scaled = StandardScaler().fit_transform(dataset)

    # Create partitions of training and testing
    # The first six columns are features and the 7th column is the target
    X_train, X_test, y_train, y_test = train_test_split(data_scaled[:, 0:6], data_scaled[:, 6],
                                                        test_size=0.25, random_state=1)

    # 3 neurons in the first hidden layer and five in the second one
    regr = MLPRegressor(hidden_layer_sizes=(3, 5), random_state=1).fit(X_train, y_train)

    # Use the predict() function with the test data to predict the values of the target variable
    prediction_result = regr.predict(X_test[2:3,:])
    print("\nPrediction Result: ", prediction_result)

    # Use the score() to compute the squared error of our model
    error = regr.score(X_test, y_test)/len(y_test)
    print("\nSquared Error: ", error)

if __name__ == "__main__":
    main()
```

6 Input Attributes: longitudinal position of the center of buoyancy, prismatic coefficient, length-displacement, beam-draught, length-beam, Froude number

1 Output Attribute: residuary resistance per unit weight of displacement

Coefficient of Determination

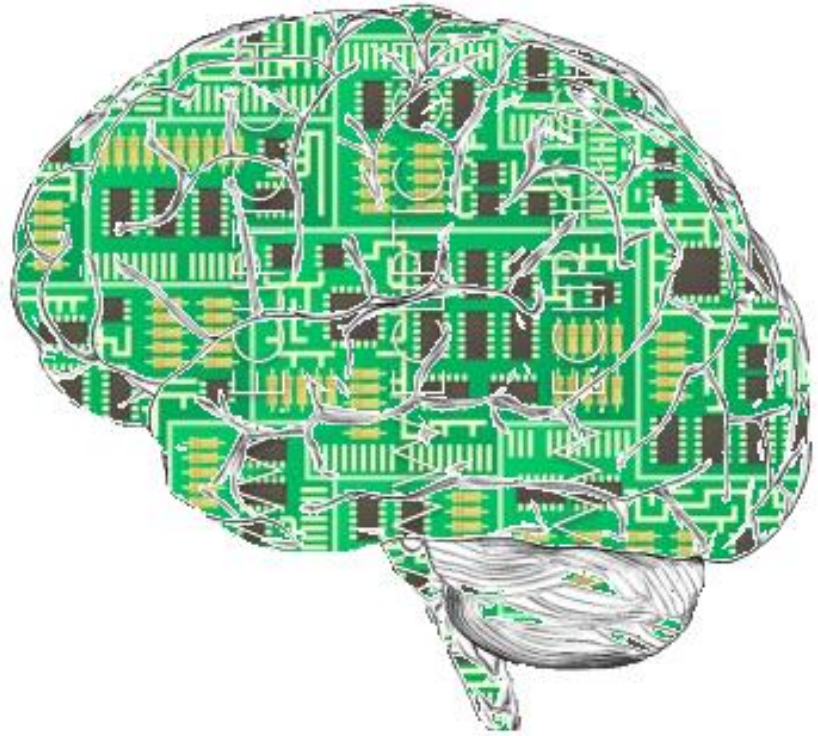
score : *float*

R^2 of `self.predict(X)` w.r.t. `y`.

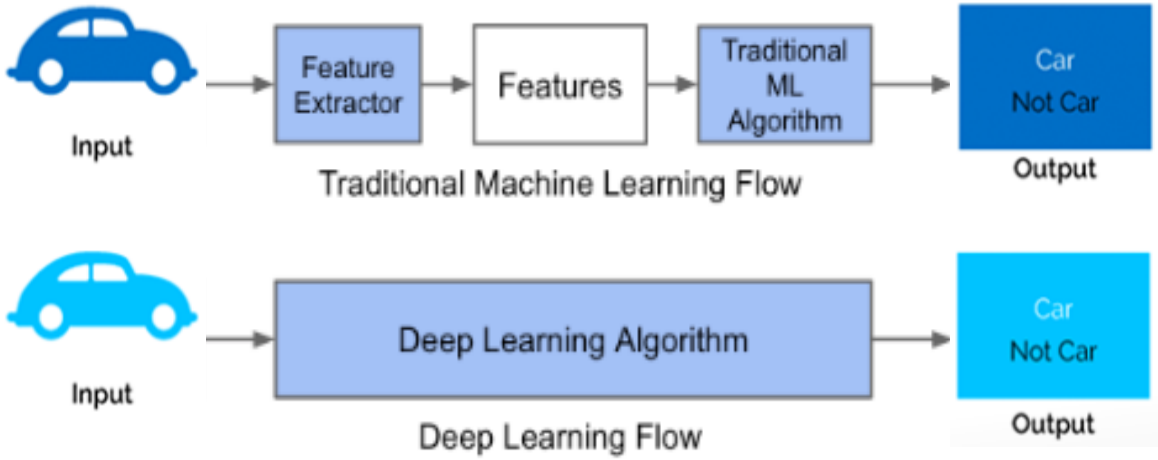
Artificial Neural Networks

https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Deep Learning (DL)

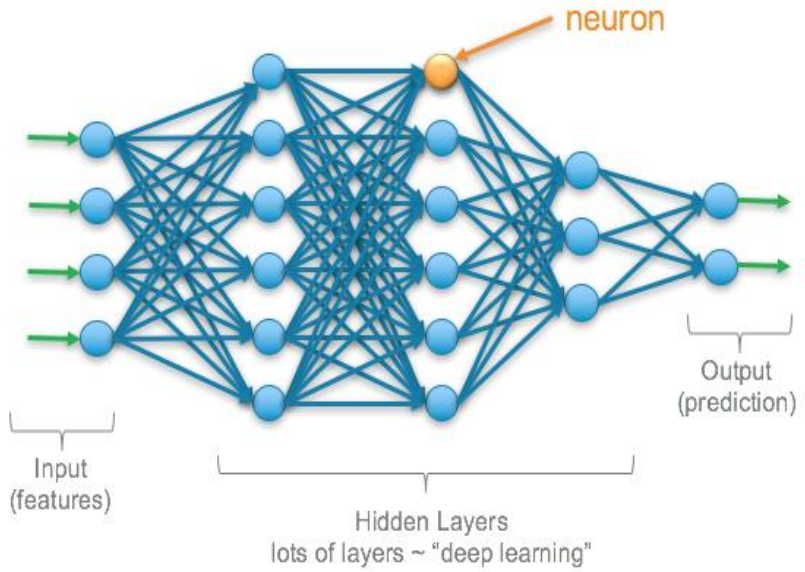


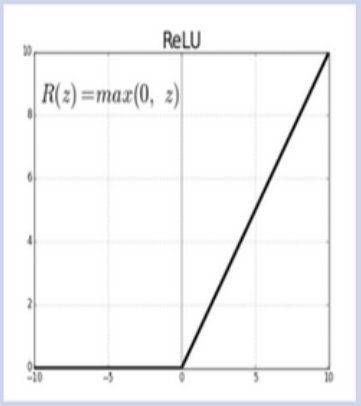
- Deep learning is a branch of machine learning based on a set of algorithms that attempt to **model high-level abstractions in data** by using multiple processing layers (Hidden and Output Layers) composed of multiple non-linear transformations (Activation Functions).
- Deep learning aims to automatically learn feature hierarchies from data as opposed to **"by hand"** feature engineering and transformation in Machine Learning.
- During the learning process, a backpropagation algorithm helps the model to fine-tune the parameters of one layer from the calculated parameters of the previous layer.
- DL is **NOT a Master Key**. DL does not and should not apply to all situations. In some cases, DL might be a very bad solution.
  - DL needs lots of data to make accurate decisions. If you do not have enough data, other ML models may be more appropriate to use.
  - DL is costly to be run because they require enormous computational power.



Deep Learning (DL): More Specifically

- Essentially, any neural network with **at least two or more** hidden layers is deep.
- There are no correct layer numbers or neuron numbers on a layer for a specific problem.
- The input to a regular ANN model is a one-dimensional matrix *R* x 1, where *R* is the number of input variables.
- In a deep neural network, tensors (i.e., *N*-dimensional matrix) are inputs.
  - One array of numbers is the **1D tensor**: [15, 2, 7, 4]
  - One Black/White Image is the **2D tensors**: a matrix indicates the color codes used in the image pixels.
  - One Color Image is the **3D tensors**: each pixel is defined using the intensity of RGB.
  - One Video is the **4D tensors**: Each video can be represented by several matrices. Each matrix represents an image sample, i.e., an additional dimension.



ReLU (Rectified Linear Unit) $R(z) = \max(0, z)$	0 to ∞	

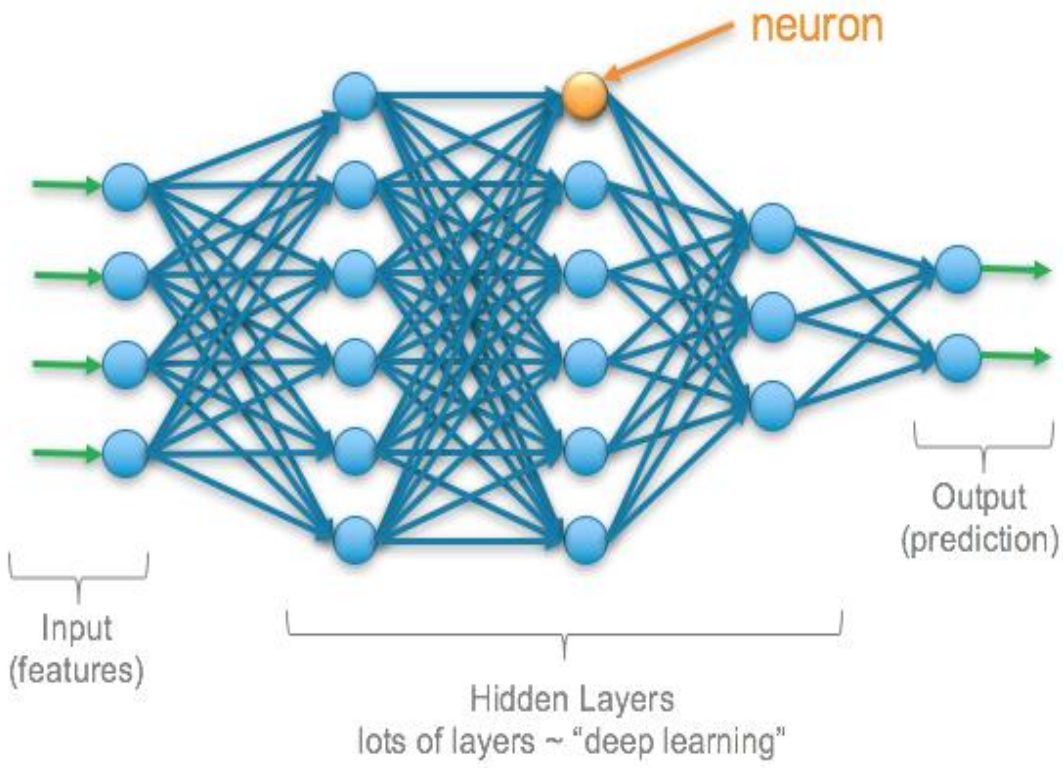
Deep Learning (DL): More Specifically

- Each training example contains values for *n* **input attributes**.

- Boolean Attributes:
  - *n* input nodes
  - 0/-1 for false and 1/+1 for true

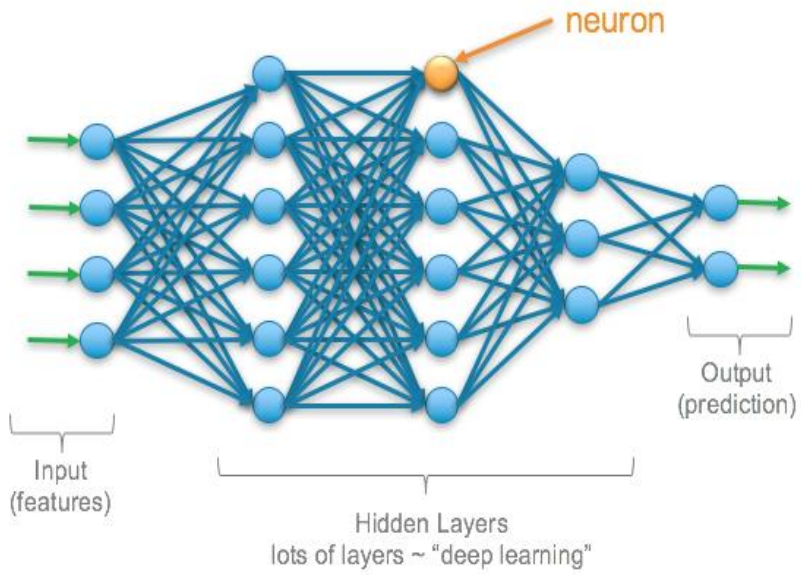
- Numeric Attributes:
  - *n* input nodes
  - Integer or Real Value

- Categorical Attributes:
  - *n* possible categorical values
  - One-hot encoding for each categorical value.
  - *n* input nodes



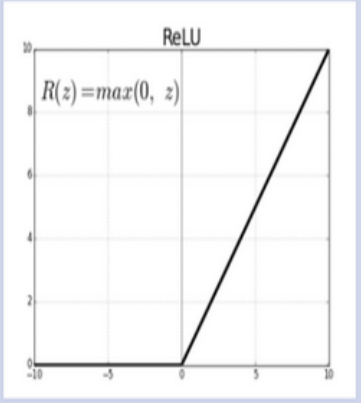
Deep Learning (DL): More Specifically

- Each **neuron** is a unit that calculates the weighted sum of the inputs from the preceding neurons and then applies a nonlinear activation function to produce its output.



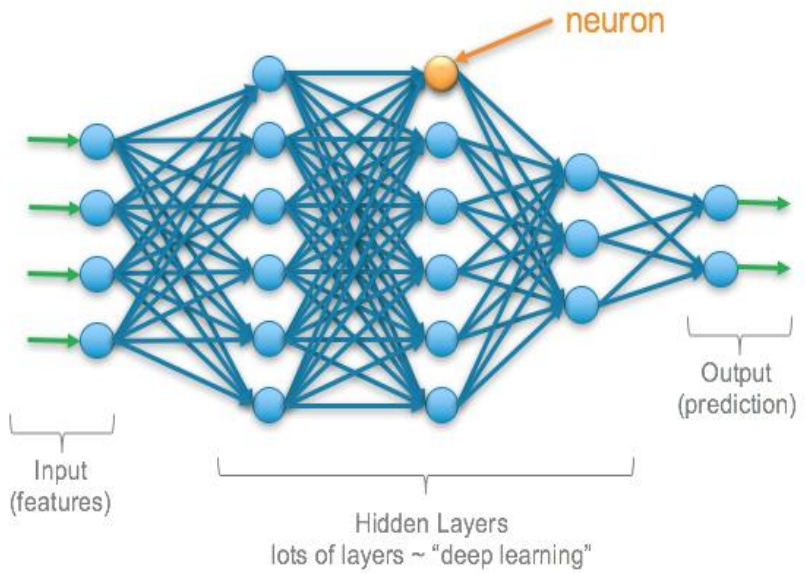
- Each **neuron** has an extra input, called a **bias** that is the constant input value 1, and a weight W_0 for that input.
  - Explain the data variance that cannot be explained by all the input features.
  - Allow the total weight of inputs to be nonzero, i.e., $W_0 \neq 0$, even when all the input feature values are zero.

- **Universal Approximation:** A network with just two hidden layers of neurons, the 1st nonlinear and the 2nd linear, can approximate **any continuous function** to an arbitrary degree of accuracy.

ReLU (Rectified Linear Unit) $R(z) = \max(0, z)$	0 to ∞	

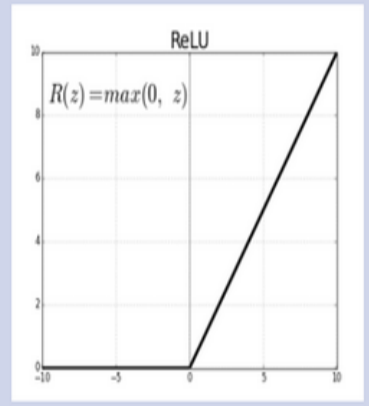
Deep Learning (DL): More Specifically

- **Rectified Linear Units (ReLU)** is the **standard** activation function for the **hidden layers** of a deep neural network.
  - Choosing the Right Activation Function: As a rule of thumb, you can begin with using **ReLU(x)** function and then move over to other activation functions in case ReLU doesn't provide with optimum results.
  - **Sigmoid(x)** function: **$1 / (1 + e^{-x})$**
  - **Softplus(x)** function: **$\ln(1 + e^{x})$**
  - **Tanh(x)** function: **$(e^{2x} - 1) / (e^{2x} + 1)$**

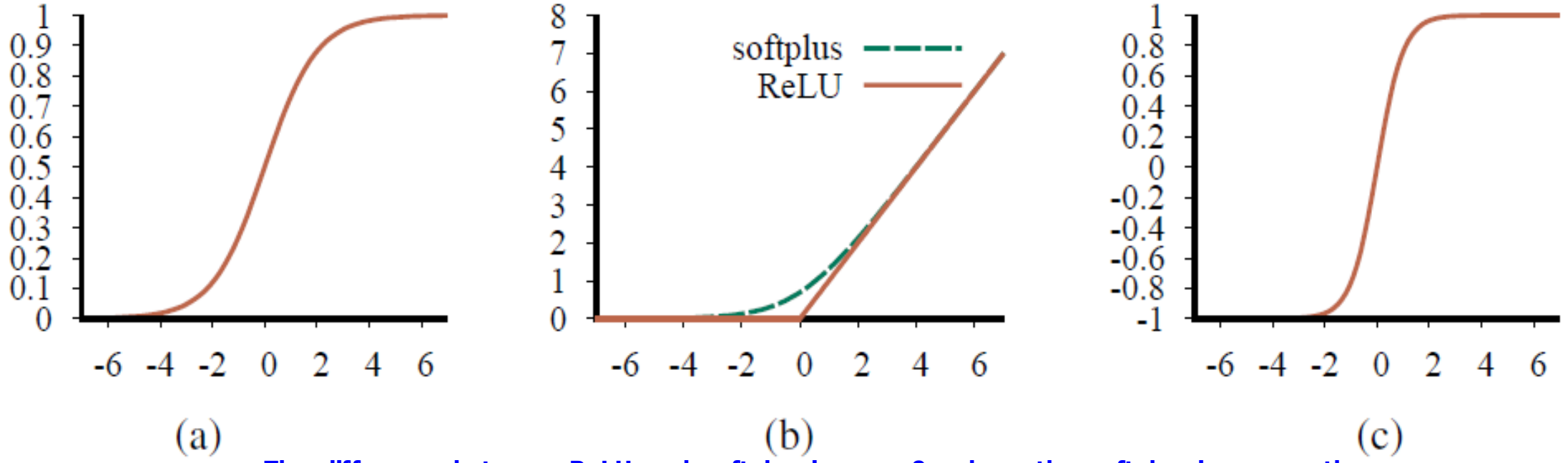


- **Common activation functions** for the **output layer** are **linear functions, sigmoid functions, and softmax functions**.
  - Regression - **Linear(x)** function: **y = x**
  - Binary Classification - **Sigmoid(x)** function:

    $1 / (1 + e^{-x})$
  - Multi-class Classification: **Softmax(x)** function: $\frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}$, where *K* is # of Classes.

ReLU (Rectified Linear Unit) $R(z) = \max(0, z)$	0 to ∞	

Deep Learning (DL): More Specifically



The difference between ReLU and softplus is near 0, where the softplus is a smooth approximation of the ReLU function. ReLU has efficient computation; however, the computation for softplus function is more expensive as it has ln and exp in its formulation.

Figure 21.2 Activation functions commonly used in deep learning systems: (a) the logistic or sigmoid function; (b) the ReLU function and the softplus function; (c) the tanh function.

$$f(x) = \frac{1}{1+e^{-x}}$$

Sigmoid or Logistic activation function

$$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$$

uation for Rectified Linear Unit(ReLU) activation fun

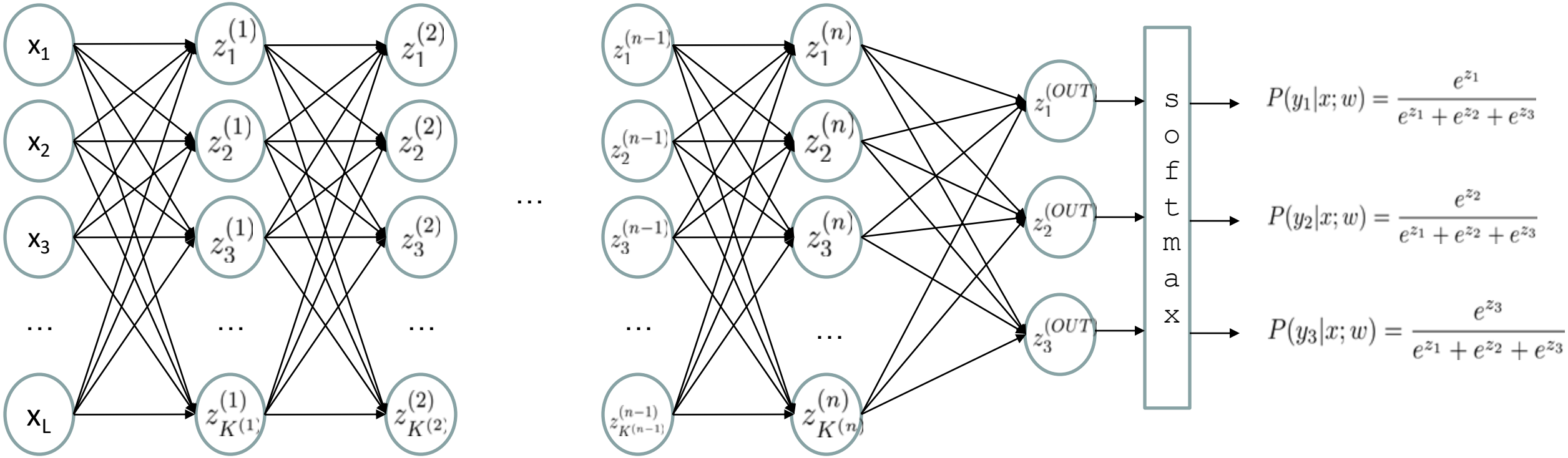
$$f(x) = \ln(1+e^{x})$$

Softplus

$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$$

Equation for Hyperbolic Tangent(TanH) activation function

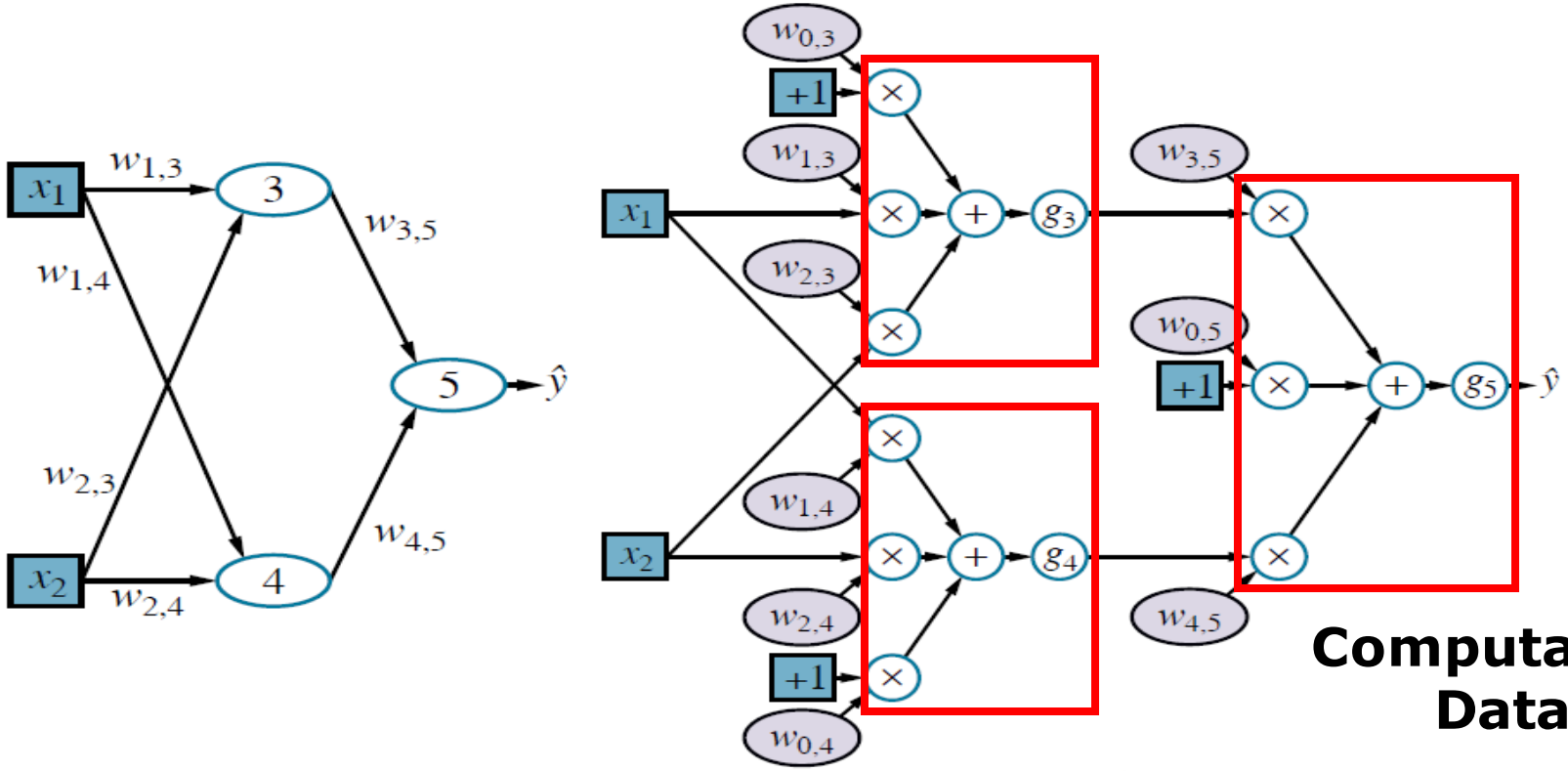
Deep Learning (DL): More Specifically



$$z_i^{(k)} = g\Big(\sum_j W_{i,j}^{(k-1,k)}\, z_j^{(k-1)}\Big)$$

g = nonlinear activation function

Deep Learning (DL): More Specifically



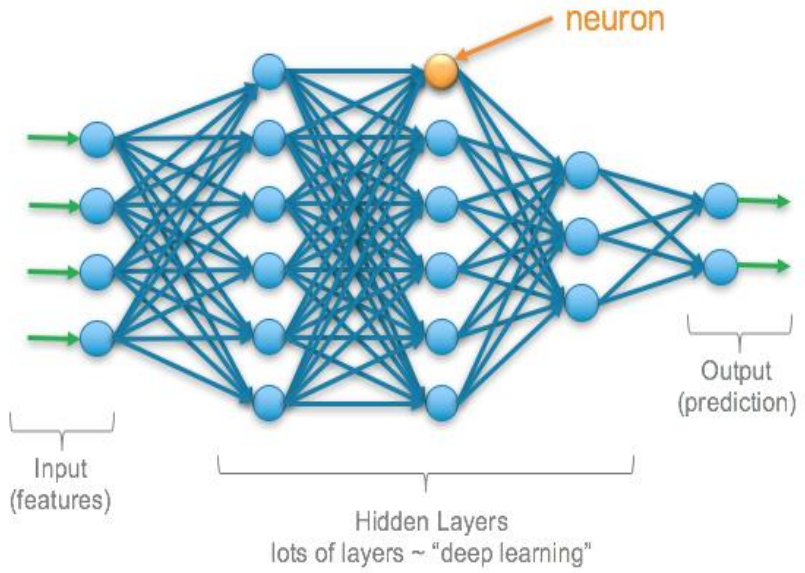
Computational Graph or Dataflow Graph

Figure 21.3 (a) A neural network with two inputs, one hidden layer of two units, and one output unit. Not shown are the dummy inputs and their associated weights. (b) The network in (a) unpacked into its full computation graph.

$$\hat{y} = g_5\left(w_{0,5} + w_{3,5}\, g_3\left(w_{0,3} + w_{1,3} x_1 + w_{2,3} x_2\right) + w_{4,5}\, g_4\left(w_{0,4} + w_{1,4} x_1 + w_{2,4} x_2\right)\right)$$

More Hidden Layers versus More Neurons?

- A feedforward network has connection only in one direction.



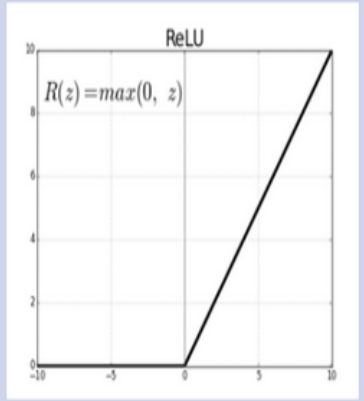
- **Question:** Why do we need deep neural network with many layers when we can include the same number of neurons in just one or few layers, i.e., a wide neural network instead of a deep neural network?

- **Answer:** It may fail to learn the underlying patterns correctly. Note that the hidden-layer is a feature-extraction mechanism that converts the original inputs to a higher-level combinations of such inputs.

  ➔ Deep and Narrow Network > Shallow and Wide Networks

  ➔ Design Network with Experience

- **Gradients and Learning Algorithms:**
  - All gradients can be computed by the method of automatic differentiation.
  - All deep learning packages provide automatic differentiation with different network structures, activation functions, loss functions, etc.
  - Adam Optimizer: Stochastic Gradient Descent (SGD)

ReLU (Rectified Linear Unit) $R(z) = \max(0, z)$	0 to ∞	

Different Types of Deep Learning Architectures

- Convolutional Neural Network
- Recurrent Neural Network
- Autoencoders
- And More

DS 541. DEEP LEARNING